

Experiment- 1

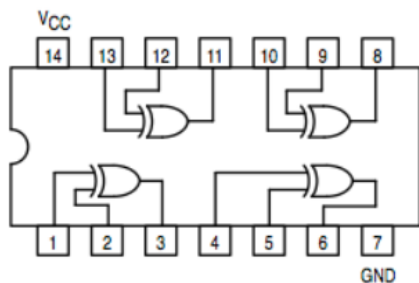
1. OBJECTIVE:

Design a 2-bit binary adder circuit using logic gates (XOR, AND, OR).

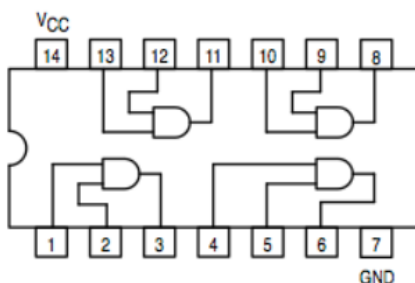
2. APPARATUS REQUIRED

Breadboard	1
DC-Power supply	1
XOR	74LS86-1
AND	74LS08-1
OR	74LS32-1
Resistors	1K Ohm-3
LED	3
Connecting wires	

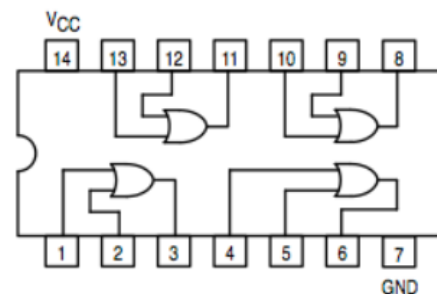
3. PIN DIAGRAM



Exclusive OR: 74LS86



AND gate: 74LS08



OR gate: 74LS32

4. THEORY

This circuit requires prerequisite knowledge of EXOR Gate, AND Gate, OR Gate, Binary Addition, and Full Adder.

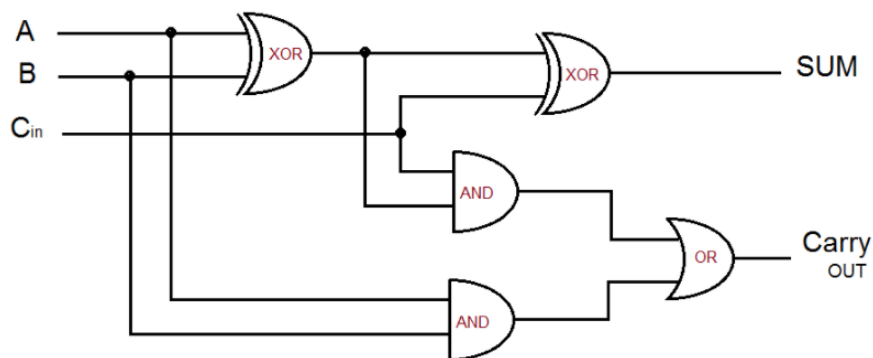


Figure: Full adder

The Boolean expression for a full adder is as follows.

For the SUM (S) bit:

$$\text{SUM} = (A \text{ XOR } B) \text{ XOR } C_{in} = (A \oplus B) \oplus C_{in}$$

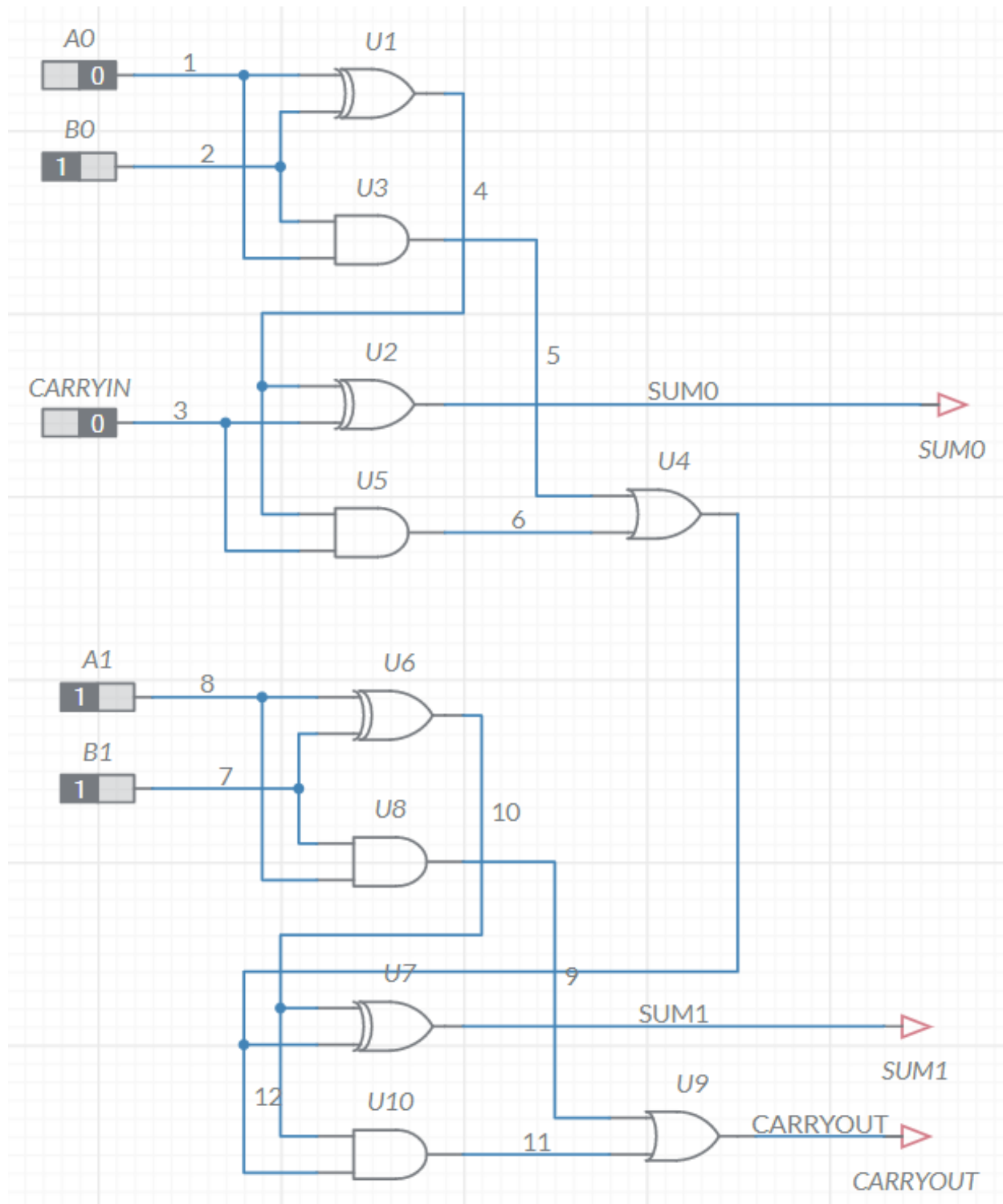
For the **CARRY-OUT** (Cout) bit:

$$\text{CARRY-OUT} = A \text{ AND } B \text{ OR } C_{in}(A \text{ XOR } B) = A.B + C_{in}(A \oplus B)$$

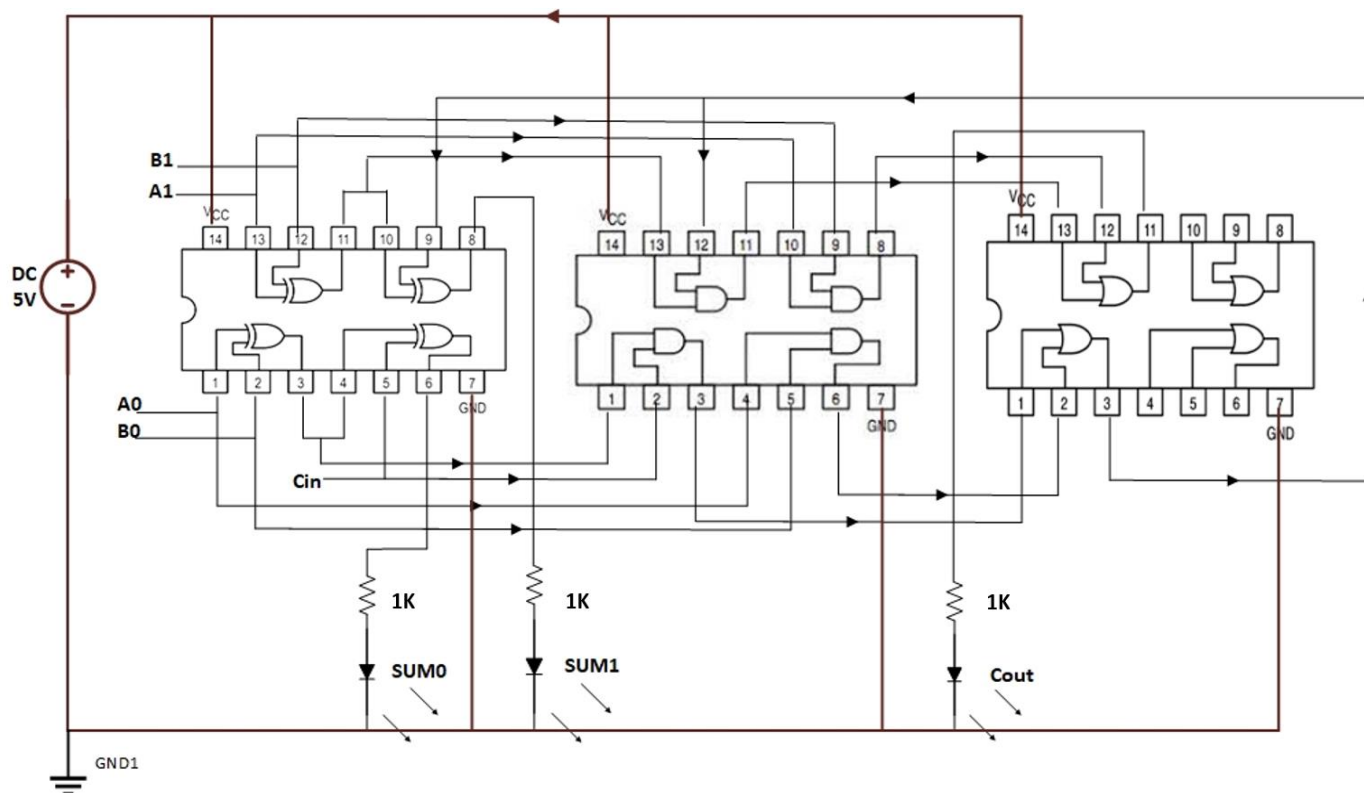
A	B	C-in	Sum	C-out
0	0	0	0	0
1	0	0	1	0
0	1	0	1	0
1	1	0	0	1
0	0	1	1	0
1	0	1	0	1
0	1	1	0	1
1	1	1	1	1

Table: Truth table of full adder

5. 2-BIT ADDER LOGIC GATE CIRCUIT



6. CIRCUIT DIAGRAM

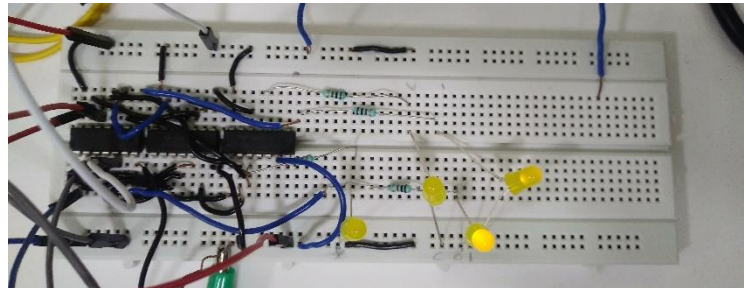


7. PROCEDURE

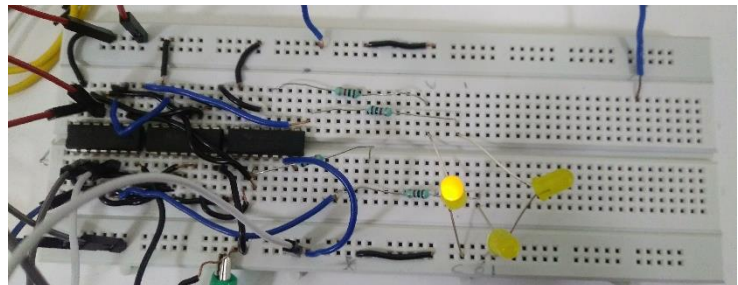
- Connect the circuit as shown in the circuit diagram.
- Check the LEDs before connecting to the circuit.
(Connect anode to positive and cathode to negative terminals of the battery)
- Don't increase the supply voltage by more than 5V.
- For logic **1** as input, connect the respective terminal to +V_{CC}.
- For logic **0** as input, connect the respective terminal to Ground.

8. EXPERIMENTAL TASK

		Carry in = 1
	A1 = 1	A0 = 0
	B1 = 0	B0 = 0
Carry out = 0	SUM0 = 1	SUM1 = 1



		Carry in = 0
	A1 = 1	A0 = 0
	B1 = 1	B0 = 0
Carry out = 1	SUM0 = 0	SUM1 = 0



Experiment -2 INSTRUMENTATION AMPLIFIER.

OBJECTIVE:

To design an instrumentation Amplifier for the given specifications using Op-Amp IC 741.

APPARATUS REQUIRED:

S.No	Name of the Apparatus	Range	Quantity
1.	Function Generator	3 MHz	1
2.	DSO	30 MHz	1
3.	2 Channel DC Power supply	0 – 30 V	1
4.	Op-Amp	IC 741	3
5.	Bread Board		1
6.	Resistors	As required	
7.	Connecting wires and probes	As required	

THEORY:

An **instrumentation** (or **instrumentational**) **amplifier** is a type of differential amplifier that has been outfitted with input buffers, which eliminate the need for input impedance matching and thus make the amplifier particularly suitable for use in measurement and test equipment. Additional characteristics include very low DC offset, low drift, low noise, very high open-loop gain, very high common-mode rejection ratio, and very high input impedances. Instrumentation amplifiers are used where great accuracy and stability of the circuit both short- and long-term are required.

The most commonly used instrumentation amplifier circuit is shown in the figure. The gain of the circuit is

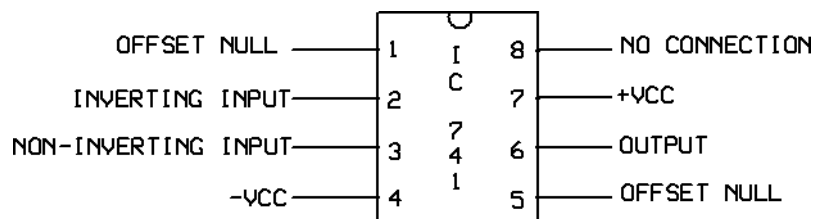
$$\frac{V_{out}}{V_2 - V_1} = \left(1 + \frac{2R_1}{R_{gain}}\right) \frac{R_3}{R_2}$$

The rightmost amplifier, along with the resistors labelled R_2 and R_3 is just the standard differential amplifier circuit, with gain = R_3 / R_2 and differential input resistance = $2 \cdot R_2$. The two amplifiers on the left are the buffers. With R_{gain} removed (open circuited), they are simple unity gain buffers; the circuit will work in that state, with gain simply equal to R_3 / R_2 and high input impedance because of the buffers. The buffer gain could be increased by putting resistors between the buffer inverting inputs and ground to shunt away some of the negative feedback; however, the single resistor R_{gain} between the two inverting inputs is a much more elegant method: it increases the differential-mode gain of the buffer pair while leaving the common-mode gain equal to 1.

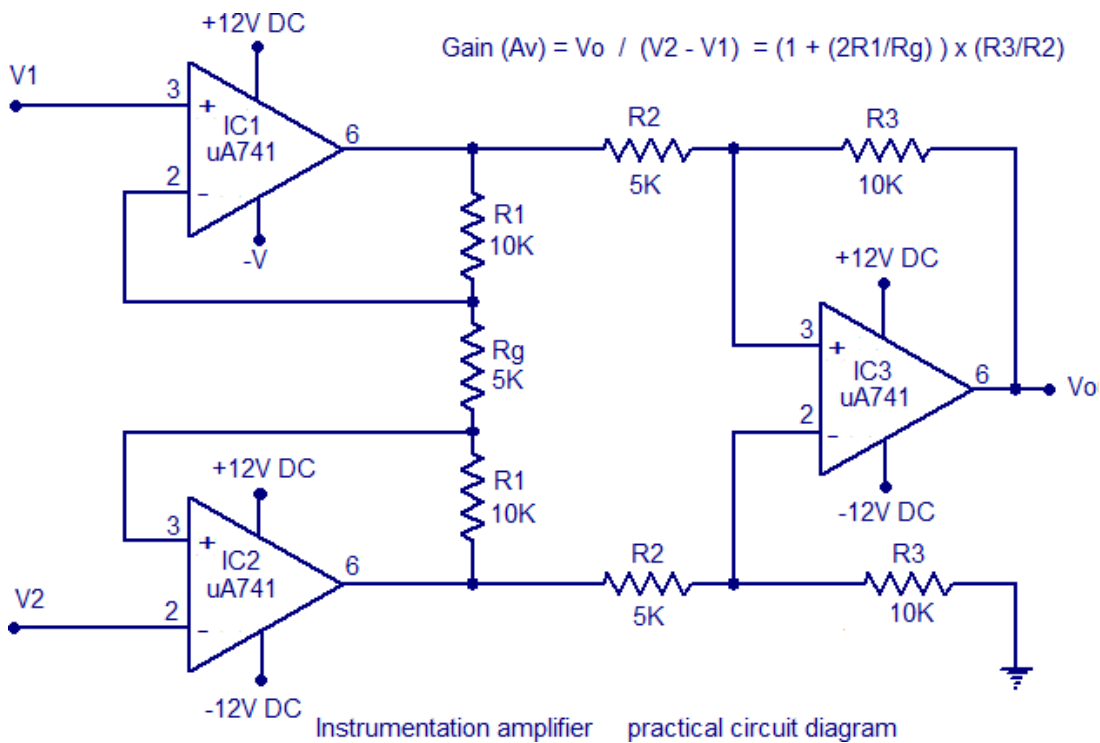
PROCEDURE:

1. Connections are given as per the circuit diagram.
2. + V_{cc} and - V_{cc} supply is given to the power supply terminal of the Op-Amp IC.
3. By adjusting the amplitude and frequency value of the sin wave in the function generator, the appropriate input voltage is applied to the input terminal of the Op-Amp.
4. The output voltage is obtained in the DSO.

PIN DIAGRAM:



CIRCUIT DIAGRAM OF INSTRUMENTATION AMPLIFIER:



Observation Table-

Input Voltage			Vo in volts	
V1 volt	V2 volt	Rg ohm	Theoretical	Practical

Result:

Output Voltage Vo

EXPERIMENT 3

OBJECTIVE: This experiment aims to understand the LTspice simulation with experimental plots (i) the transfer characteristics of n-channel and p-channel enhancement-type MOSFETs (ii) the output characteristics of n-channel and p-channel enhancement-type MOSFETs, and (iii) the voltage transfer characteristics (VTC) of a CMOS inverter by connecting the n-MOSFET and p-MOSFET.

APPARATUS: NMOS (BS 170), PMOS (BS 250), bread board, resistors ($1\text{K}\Omega$), connecting wires, Ammeters (0-10mA/ 0-25mA), DC power supply (0-12V) and multimeters.

CIRCUIT DIAGRAM:

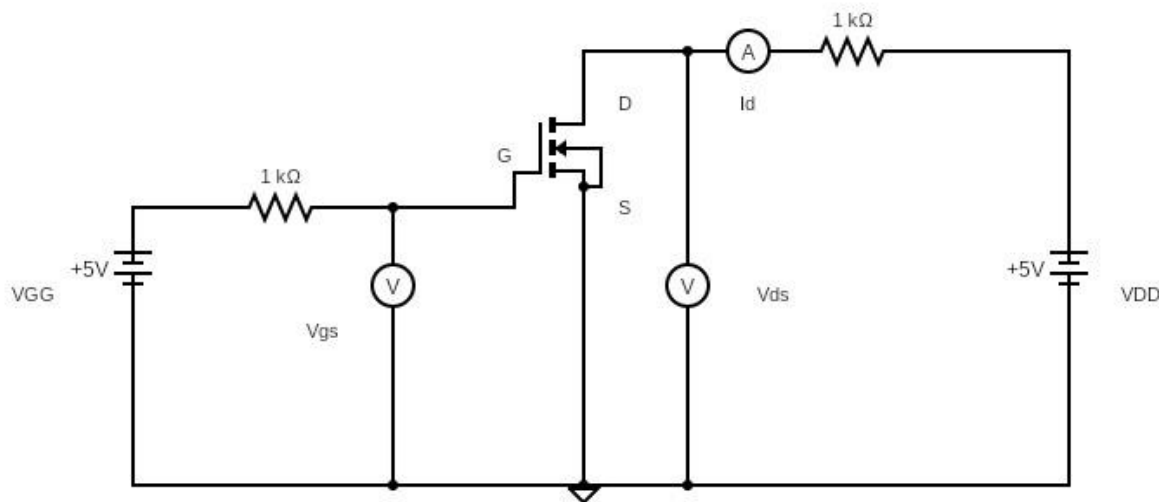


Figure 1: Circuit diagram of NMOS

1. **Cut-Off Region**

Cut-off region is a region in which the MOSFET will be OFF as there will be negligible current flow through it.

2. **Ohmic or Linear Region**

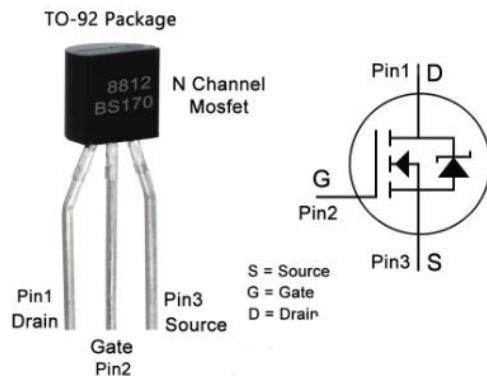
Ohmic or linear region is a region wherein the current I_D increases with an increase in the value of V_{DS} .

3. **Saturation Region**

In saturation region, the drain current I_D is quite constant irrespective of an increase in V_{DS} .

MOSFET	Cut-Off	Ohmic/Linear	Saturation
n-channel Enhancement-type	$V_{GS} < V_T$	$V_{GS} > V_T$ and $V_{DS} < V_{GS} - V_T$	$V_{GS} > V_T$ and $V_{DS} > V_{GS} - V_T$
p-channel Enhancement-type	$V_{GS} > -V_T$	$V_{GS} < -V_T$ and $V_{DS} > V_{GS} - V_T$	$V_{GS} < -V_T$ and $V_{DS} < V_{GS} - V_T$

BS170 MOSFET Pinout



BS250 MOSFET Pinout

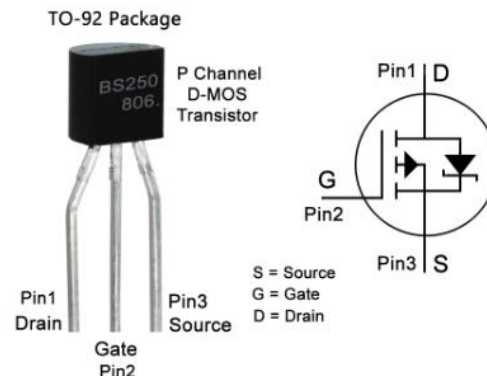


Figure 2: Pin diagram of BS170 and BS250 MOSFETS

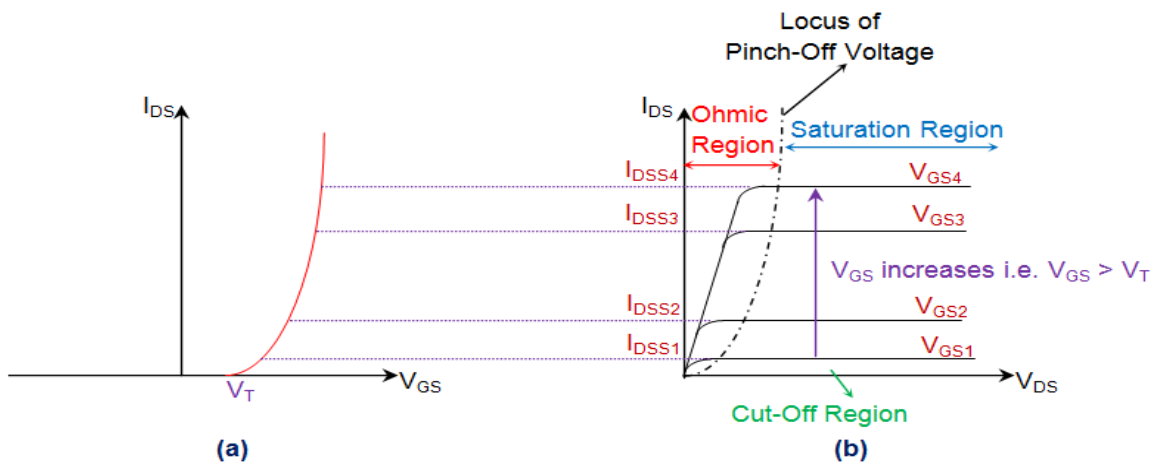


Figure 3: (a) Transfer characteristics of NMOS (b) Output characteristic of NMOS

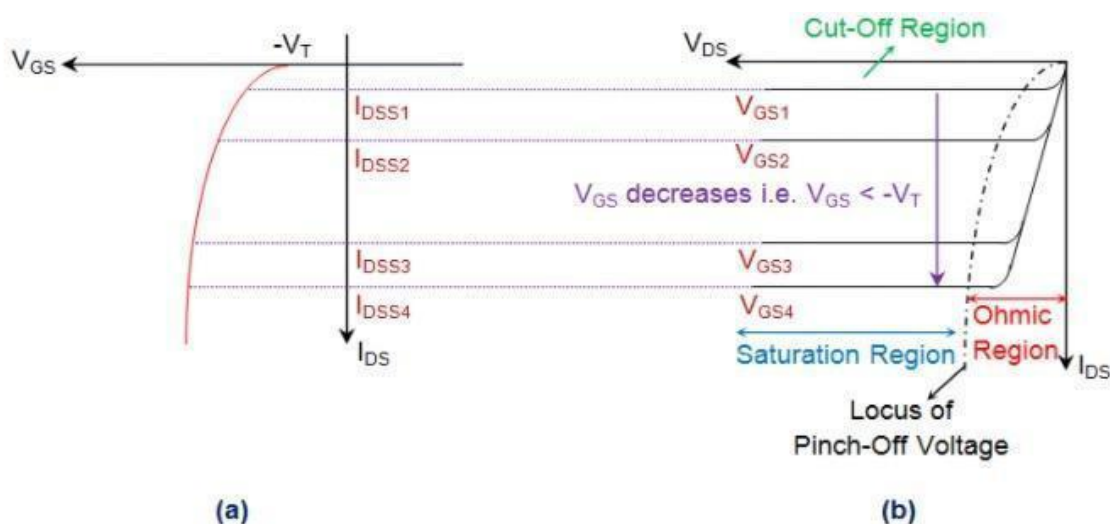


Figure 4:(a) Transfer characteristic of NMOS (b) Output characteristic of PMOS

PROCEDURE:**TRANSFER CHARACTERISTICS:** V_{GS} vs. I_D for different V_{DS} .

1. Connect the circuit as per given diagram properly.
2. Set the voltage V_{DS} as constant at 3 V, 5 V and 10 V.
3. Vary V_{GS} in the step of 0.5 V from 0 to 11 V and note down value of drain currents I_D . Tabulate all the readings.
4. Plot the output characteristics V_{DS} vs I_D (for different values of V_{GS}) and transfer characteristics V_{GS} vs I_D .

OUTPUT CHARACTERISTICS: V_{DS} vs. I_D for different V_{GS} .

1. Connect the circuit as per given diagram properly.
2. Keep V_{GS} as constant to 1 V and sweep V_{DS} and take readings of I_D . Vary V_{DS} in step of 1 V up to 11 V and measure the drain currents I_D . Tabulate all the readings.
3. Repeat the above procedure for V_{GS} as 1 V, 2 V, 2.5 V, 3 V, 3.5 V, 4 V, 4.5 V and 5 V.

OBSERVATION TABLE:**TRANSFER CHARACTERISTICS**

V_{GS} (V)	$V_{DS} = 5\text{ V}$	$V_{DS} = 10\text{ V}$	V_{GS} (V)	$V_{DS} = 5\text{ V}$	$V_{DS} = 10\text{ V}$
	I_D (mA)	I_D (mA)		I_D (mA)	I_D (mA)
0			6		
0.5			6.5		
1			7		
1.5			7.5		
2			8		
2.5			8.5		
3			9		
3.5			9.5		
4			10		
4.5			10.5		

5			11		
5.5					

OUTPUT CHARACTERISTICS

V_{DS} (V)	$V_{GS} = 1\text{ V}$	$V_{GS} = 2\text{ V}$	$V_{GS} = 2.5\text{ V}$	$V_{GS} = 3\text{ V}$	$V_{GS} = 4\text{ V}$	$V_{GS} = 4.5\text{ V}$	$V_{GS} = 5\text{ V}$
	I_D (mA)	I_D (mA)	I_D (mA)	I_D (mA)	I_D (mA)	I_D (mA)	I_D (mA)
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							

CALCULATIONS: Do for both NMOS and PMOS.

- Threshold voltage V_T :** Gate to source voltage at which, drain current starts flowing
- Output drain resistance:** It is given by the relation of small change in drain to source voltage (ΔV_{DS}) to the corresponding change in Drain Current (ΔI_D) for a constant V_{GS} .

$$r_d \text{ or } r_o = \Delta V_{DS} / \Delta I_D \text{ at a constant } V_{GS}$$

- Two-port small signal parameters:** Assuming $i_g = 0$ in the two-port network, determine g_m and g_d where $i_d = g_m v_g + g_d v_d$

- Transconductance g_m :** Ratio of small change in drain current (ΔI_D) to the corresponding change in gate to source voltage (ΔV_{GS}) for a constant V_{DS} .

$$g_m = \Delta I_D / \Delta V_{GS} \text{ at constant } V_{DS}$$

- Drain conductance g_d :** Ratio of small change in drain current (ΔI_D) to the corresponding change in gate to drain voltage (ΔV_{DS}) for a constant V_{GS} .

$$g_d = \Delta I_D / \Delta V_{DS} \text{ at constant } V_{GS}$$

RESULTS:

- V_T : _____
- g_m : _____
- g_d : _____
- r_o : _____

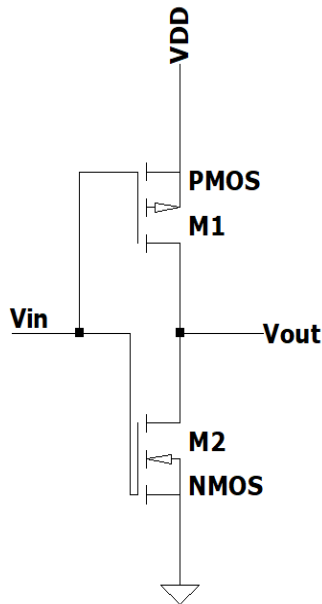
CMOS INVERTER:**Circuit Diagram:**

Figure 5: CMOS Inverter

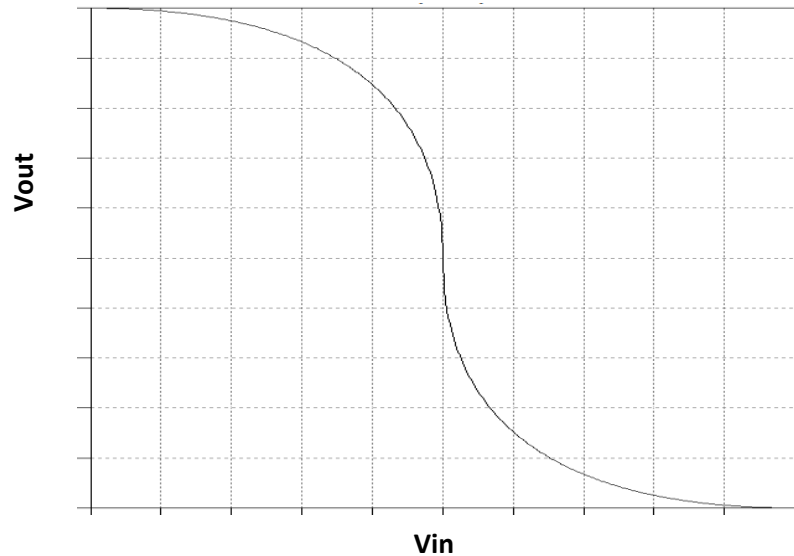


Figure 6: Voltage transfer characteristics

THEORY: A CMOS inverter consists of a PMOS and an NMOS transistor connected in series. The drain and gate terminals of the two transistors are connected. The supply voltage V_{DD} is connected to PMOS source terminal and the NMOS source terminal is connected to ground. The voltage V_{in} is applied at the gate terminals and V_{out} is the output voltage at the drain terminals. The input/output characteristics of a CMOS inverter is called voltage transfer characteristics and it has a sharp transition region, which makes it energy-efficient during switching between low and high voltage levels. At low input voltage circuit output high voltage and vice-versa, thereby it acts as an inverter.

Type of MOSFET	Condition of MOSFET	State of MOSFET
NMOS	$V_{GS} < V_{Tn}$	OFF
NMOS	$V_{GS} > V_{Tn}$	ON
PMOS	$V_{GS} < -V_{Tp}$	OFF
PMOS	$V_{GS} > -V_{Tp}$	ON

PRECAUTION

1. Before performing the experiment, check the power supply, bread board, connecting wires, multimeter mode.

SPICE SIMULATIONS

You would need to match the SPICE simulation results versus experimental measurements. You may need to change the relevant model parameters to match your measurements.

Transfer characteristics of NMOS (V_{GS} vs I_D at constant V_{DS})

1. Connect the circuit as shown in Figure 7.
2. Select component on menu bar, search *nmos* for n-channel MOSFET.
3. Click on *.op* copy the BS170 model and select OK.
4. Tab the model parameters in the schematic window.
5. Rename the component model name NMOS to BS170.
6. Click on *Simulate*, select *Edit Simulation Command* and then click on *DC Sweep*, select *1st Source*.
7. Enter in *Source to Sweep: V1*, *Type of sweep: Linear*, *Start value: 0*, *Stop value: 3*, *Increment: 0.01*.
8. Select *2nd Source*. Enter in *Source to Sweep: V2*, *Type of sweep: Linear*, *Start value: 5*, *Stop value: 10*, *Increment: 5*.
9. Run the simulation.
10. You will get V_1 (V_{GG}) in x -axis.
11. Right click on x -axis, change V_1 by $V(N004)$ (V_{GS} , *gate – source voltage*) in quantity plotted window, then click on drain current (I_D) for transfer characteristics.

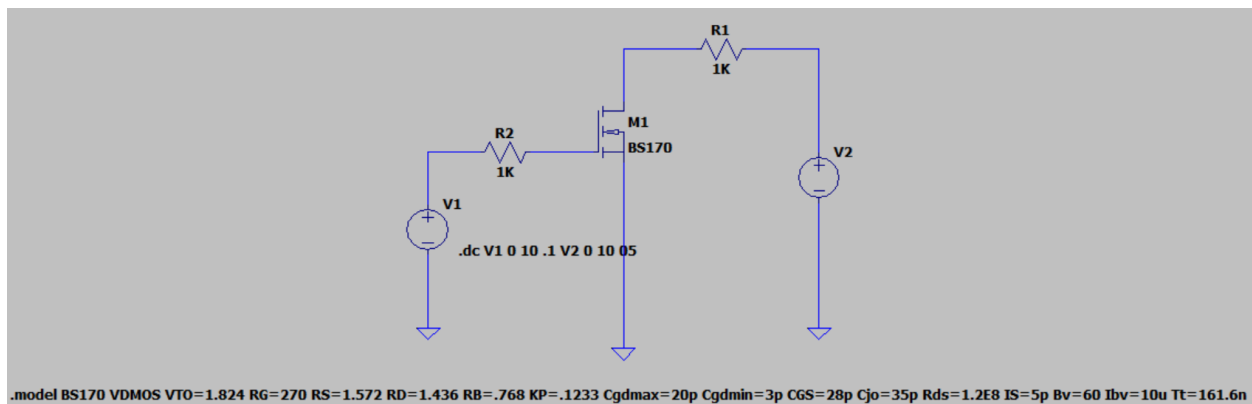


Figure 7: LTspice schematic of NMOS transfer characteristics circuit

Output characteristics of NMOS (V_{CE} vs I_C at constant V_{BB})

1. Connect the circuit as shown in Figure 8.
2. Click on *Simulate*, select *Edit Simulation Command* and then click on *DC Sweep*, select *1st Source*.

3. Enter in *Source to Sweep*: V2, *Type of sweep*: Linear, *Start value*: 0, *Stop value*: 3, *Increment*: 0.2.
4. Select 2nd *Source*. Enter in *Source to Sweep*: V1, *Type of sweep*: Linear, *Start value*: 0, *Stop value*: 1, *Increment*: 1.
5. Run the simulation.
6. You will get V2 (V_{DD}) in x -axis.
7. Right click on x -axis, change V2 by V(N001) (V_{DS} , *drain-source voltage*) in quantity plotted window, then click on drain current (I_D).

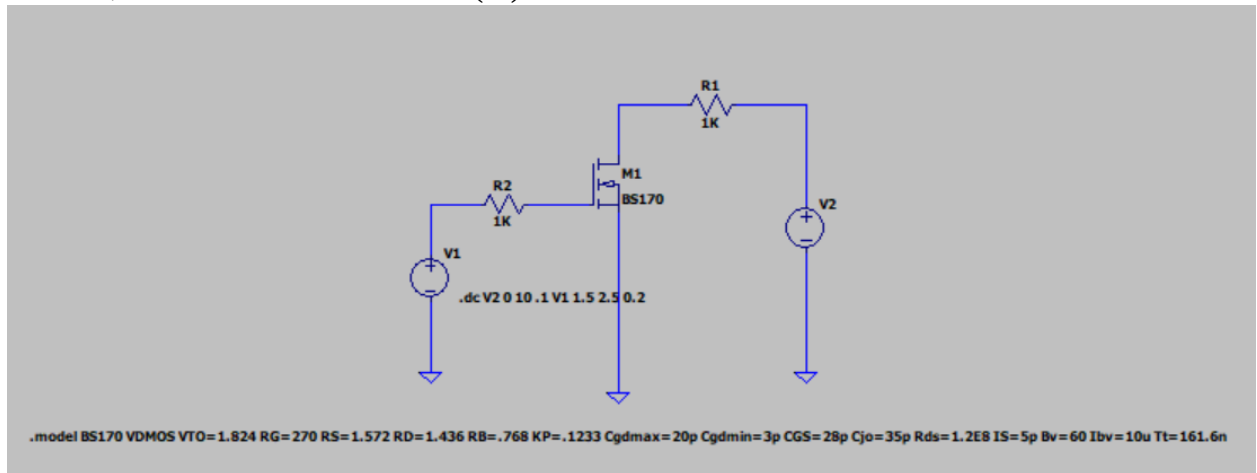


Figure 8: LTspice schematic of NMOS output characteristics circuit

Transfer characteristics of PMOS (V_{GS} vs I_D at constant V_{DS})

12. Connect the circuit as shown in Figure 9.
13. Select component on menu bar, search *pmos* for p-channel MOSFET.
14. Click on *.op* copy the BS250 model and select OK.
15. Tab the model parameters in the schematic window.
16. Rename the component model name PMOS to BS250P.
17. Click on *Simulate*, select *Edit Simulation Command* and then click on *DC Sweep*, select *1st Source*.
18. Enter in *Source to Sweep*: V1, *Type of sweep*: Linear, *Start value*: 0, *Stop value*: 3, *Increment*: 0.01.
19. Select 2nd *Source*. Enter in *Source to Sweep*: V2, *Type of sweep*: Linear, *Start value*: 5, *Stop value*: 10, *Increment*: 5.
20. Run the simulation.
21. You will get V1 (V_{GG}) in x -axis.
22. Right click on x -axis, change V1 by V(N004) (V_{GS} , *gate – source voltage*) in quantity plotted window, then click on drain current (I_D) for transfer characteristics.

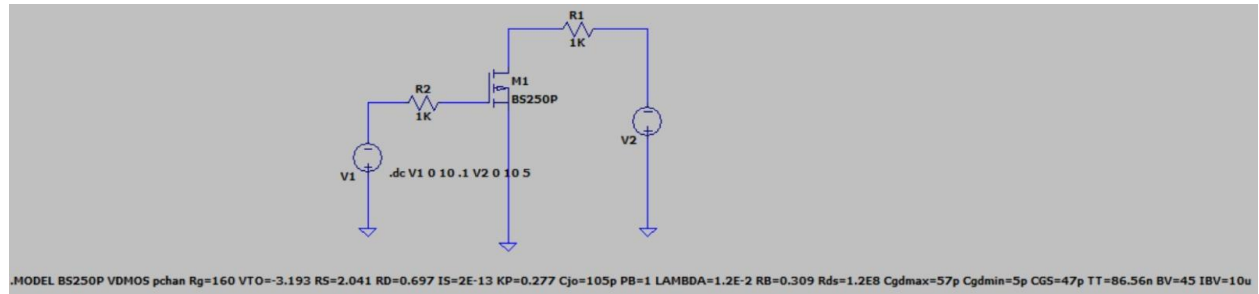


Figure 9: LTspice schematic of PMOS transfer characteristics circuit

Output characteristics of NMOS (V_{CE} vs I_C at constant V_{BB})

8. Connect the circuit as shown in Figure 10.
9. Click on *Simulate*, select *Edit Simulation Command* and then click on *DC Sweep*, select *1st Source*.
10. Enter in *Source to Sweep*: V2, *Type of sweep*: *Linear*, *Start value*: 0, *Stop value*: 3, *Increment*: 0.2.
11. Select *2nd Source*. Enter in *Source to Sweep*: V1, *Type of sweep*: *Linear*, *Start value*: 0, *Stop value*: 1, *Increment*: 1.
12. Run the simulation.
13. You will get V2 (V_{DD}) in x -axis.
14. Right click on x -axis, change V2 by V(N001) (V_{DS} , *drain-source voltage*) in quantity plotted window, then click on drain current (I_D).

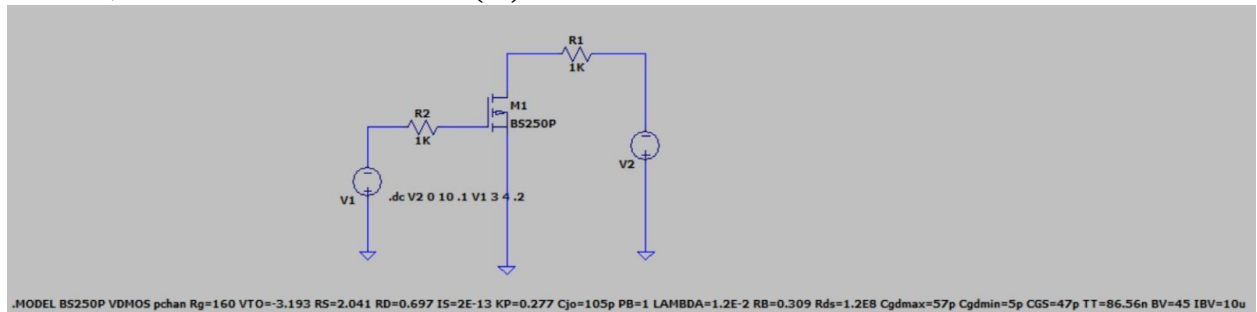


Figure 10: LTspice schematic of PMOS output characteristics circuit

EXPERIMENT 4

ARDUINO – I

Objectives

- [1]. Blink the on-board LED.
- [2]. Blink an external LED.
- [3]. Blink LED (inbuilt and external) without time-delay function.
- [4]. Read analog value of voltage by connecting a 10 k Ω pot.
- [5]. Vary the brightness of an external LED through a pot (10 k Ω).

Theory

Arduino

Arduino is an open-source electronics platform based on easy-to-use hardware and software. Arduino boards are able to read inputs - light on a sensor, a finger on a button, or a Twitter message - and turn it into an output - activating a motor, turning on an LED, publishing something online. You can tell your board what to do by sending a set of instructions to the microcontroller on the board. To do so you use the Arduino programming language (based on Wiring), and the Arduino Software (IDE), based on Processing.

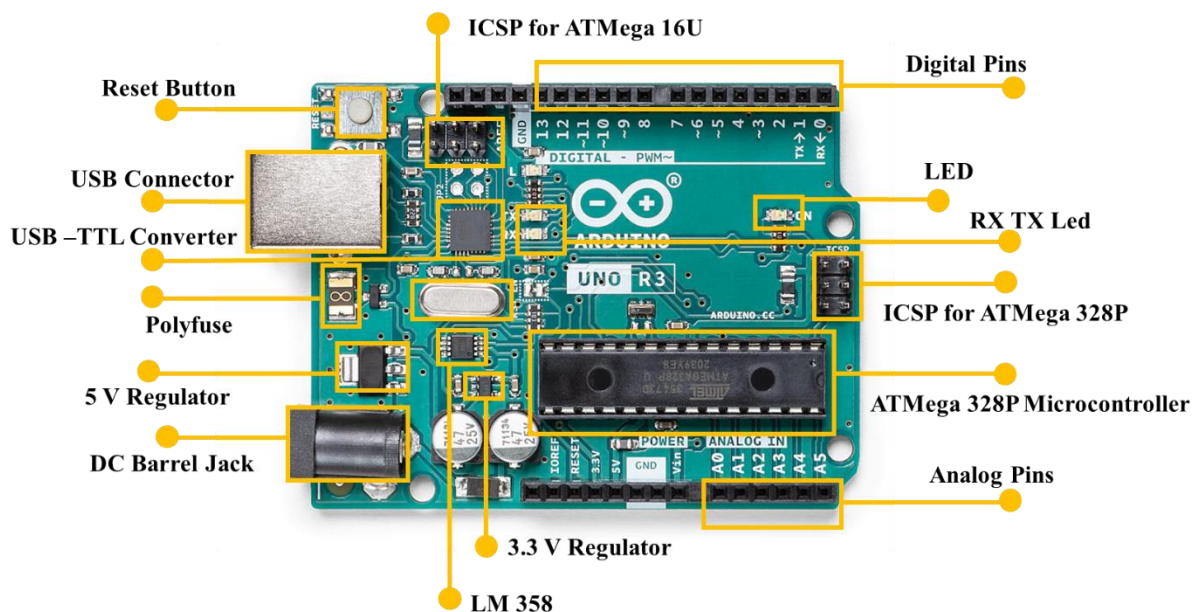


Figure 1. An Arduino board (UNO R3) and components of Arduino

How to connect to an Arduino board

- a. Install ARDUINO IDE software on your PC.
- b. Connect the Arduino board with your PC through USB cable.
- c. Run ARDUINO IDE software on your PC.
- d. Click on *tools* option as shown in Figure 2.
- e. Select the board.
- f. Select the port.
- g. Now your board is ready to do programming.

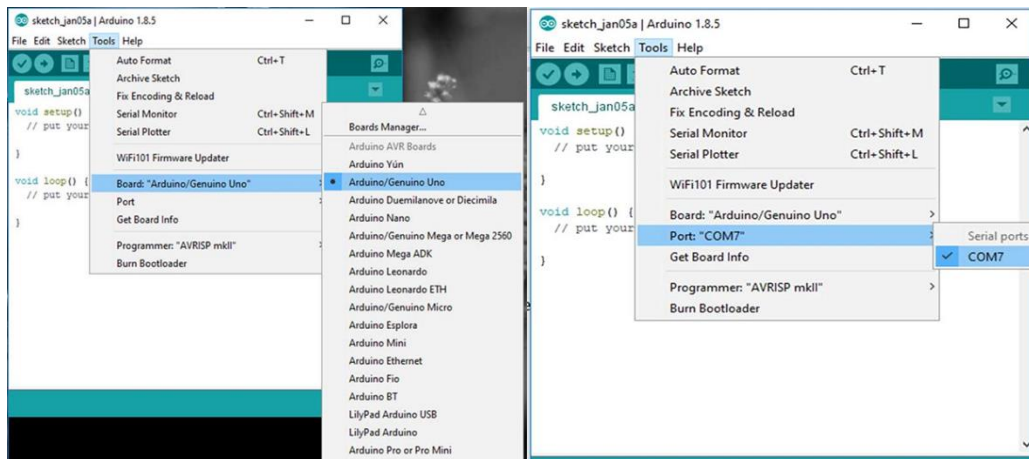


Figure 2. Interfacing of an Arduino board with PC.

Objective 1: Blink the on-board LED.**Hardware Requirement:** An Arduino board, USB Cable.**Code:** For code refer Figure 3.

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000);                     // wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off by making the voltage LOW
  delay(1000);                     // wait for a second
}
```

Figure 3. Code for blink the on-board LED.

Objective 2: Blink an external on-board LED.**Hardware Requirement:**

- An Arduino Board
- USB Cable
- A LED
- A resistor (220 Ω , etc.)
- Jumper Wires

Hardware connection

- Connect the positive side of your LED (longer leg) to Arduino digital Pin 13 (or another digital pin, don't forget to change the code to Match).
- Connect the negative side of your LED (shorter leg) to a 220 Ω resistor. Connect the other side of the resistor to ground.

- c. Always use resistors between the Arduino and LEDs to keep the LEDs from burning out due to too much current.
- d. Upload your code to Arduino and the led starts blinking.

Code: For code refer Figure 4.

```
void setup()
{
    //Initialize the digital pin as an output.
    pinMode(13,OUTPUT);
    Serial.begin(9600);
}
void loop()
{
    digitalWrite(13, HIGH);
    delay(1000);
    digitalWrite(13, LOW);
    delay(1000);
}
```

Figure 4. Code for blink an external LED.

Objective 3: Blink (inbuilt and external) LED without time delay function.

Hardware Requirement:

- a. An Arduino Board
- b. USB Cable
- c. A LED
- d. A resistor (220 Ω , etc.)
- e. Jumper Wires

Hardware connection for an external LED

- a. Connect the positive side of your LED (longer leg) to Arduino digital Pin 13 (or another digital pin, don't forget to change the code to Match).
- b. Connect the negative side of your LED (shorter leg) to a 220 Ω resistor. Connect the other side of the resistor to ground.
- c. Always use resistors between the Arduino and LEDs to keep the LEDs from burning out due to too much current.
- d. Upload your code to Arduino and the led starts blinking.

Code:

- a. *To blink inbuilt LED without time delay function code, refer Figure 5.*
- b. *Use understanding from previous experiments to blink an external LED without time delay function code.*

```

// constants won't change. Used here to set a pin number:
const int ledPin = LED_BUILTIN; // the number of the LED pin

// Variables will change:
int ledState = LOW; // ledState used to set the LED

// Generally, you should use "unsigned long" for variables that hold time
// The value will quickly become too large for an int to store
unsigned long previousMillis = 0; // will store last time LED was updated

// constants won't change:
const long interval = 1000; // interval at which to blink (milliseconds)

void setup() {
  // set the digital pin as output:
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // here is where you'd put code that needs to be running all the time.

  // check to see if it's time to blink the LED; that is, if the difference
  // between the current time and last time you blinked the LED is bigger than
  // the interval at which you want to blink the LED.
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
    // save the last time you blinked the LED
    previousMillis = currentMillis;

    // if the LED is off turn it on and vice-versa:
    if (ledState == LOW) {
      ledState = HIGH;
    } else {
      ledState = LOW;
    }

    // set the LED with the ledState of the variable:
    digitalWrite(ledPin, ledState);
  }
}

```

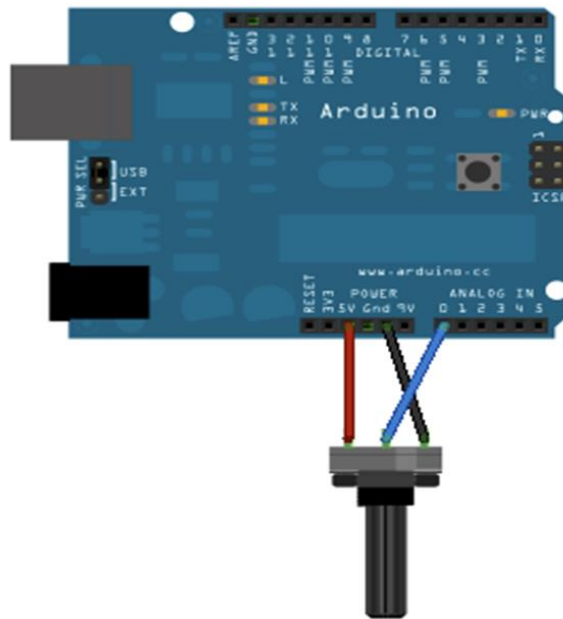
Figure 5. An Arduino program to blink inbuilt LED without time delay function.

Objective 4: Read analog value of voltage by connecting a 10 k Ω pot.

Hardware Requirement:

- An Arduino Board
- USB Cable
- A pot of 10 k Ω
- Jumper Wires
- A Breadboard

Circuit: For connection refer Figure 6.

Figure 6. Connection of the 10 k Ω pot with Arduino

Connect the three wires from the potentiometer to your board. The first goes to ground from one of the outer pins of the potentiometer. The second goes to 5 volts from the other outer pin of the potentiometer. The third goes from the middle pin of the potentiometer to analog input 0. By turning the shaft of the potentiometer, you change the amount of resistance on either side of the wiper which is connected to the center pin of the potentiometer. This changes the voltage at the center pin. When the resistance between the center and the side connected to 5 volts is close to zero (and the resistance on the other side is close to 10 k Ω), the voltage at the center pin nears 5 volts. When the resistances are reversed, the voltage at the center pin nears 0 volts, or ground. This voltage is the analog voltage that you're reading as an input. The microcontroller of the board has a circuit inside called an analog-to-digital converter *or* ADC that reads this changing voltage and converts it to a number between 0 and 1023. When the shaft is turned all the way in one direction, there are 0 volts going to the pin, and the input value is 0. When the shaft is turned all the way in the opposite direction, there are 5 volts going to the pin and the input value is 1023. In between, `analogRead()` returns a number between 0 and 1023 that is proportional to the amount of voltage being applied to the pin.

Code: For code refer Figure 7.

```
// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
}

// the loop routine runs over and over again forever:
void loop() {
  // read the input on analog pin 0:
  int sensorValue = analogRead(A0);
  // Convert the analog reading (which goes from 0 - 1023) to a voltage (0 - 5V):
  float voltage = sensorValue * (5.0 / 1023.0);
  // print out the value you read:
  Serial.println(voltage);
}
```

Figure 7. An Arduino program to read analog value of voltage by connecting a 10 k Ω pot.

Objective 5: Vary the brightness of an external LED through a pot (10 k Ω).**Hardware Requirement:**

- An Arduino Board
- USB Cable
- A pot of 10 k Ω
- A LED
- Jumper Wires
- A Breadboard

Circuit: For circuit refer Figure 8.

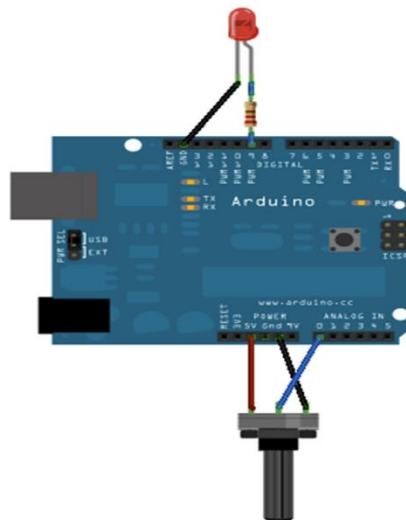


Figure 8. Connection diagram of an external LED and a pot to control the brightness of LED.

In the sketch above, after declaring two pin assignments (analog 0 for our potentiometer and digital 9 for your LED) and two variables, *sensorValue* and *outputValue*, the only things that you do in the *setup()* function is to begin serial communication.

Next, in the main loop, *sensorValue* is assigned to store the raw analog value read from the potentiometer. Arduino has an *analogRead* range from 0 to 1023, and an *analogWrite* range only from 0 to 255, therefore the data from the potentiometer needs to be converted to fit into the smaller range before using it to dim the LED.

In order to convert this value, use a function called *map()*:

```
outputValue = map(sensorValue, 0, 1023, 0, 255);
```

outputValue is assigned to equal the scaled value from the potentiometer. *map()* accepts five arguments: The value to be mapped, the low range and high values of the input data, and the low and high values for that data to be remapped to. In this case, the sensor data is mapped down from its original range of 0 to 1023 to 0 to 255.

The newly mapped sensor data is then output to the *analogOutPin* dimming or brightening the LED as the potentiometer is turned. Finally, both the raw and scaled sensor values are sent to the Arduino Software (IDE) serial monitor window, in a steady stream of data.

Code: For code refer Figure 9.

```
// These constants won't change. They're used to give names to the pins used:
const int analogInPin = A0; // Analog input pin that the potentiometer is attached to
const int analogOutPin = 9; // Analog output pin that the LED is attached to

int sensorValue = 0; // value read from the pot
int outputValue = 0; // value output to the PWM (analog out)

void setup() {
  // initialize serial communications at 9600 bps:
  Serial.begin(9600);
}

void loop() {
  // read the analog in value:
  sensorValue = analogRead(analogInPin);
  // map it to the range of the analog out:
  outputValue = map(sensorValue, 0, 1023, 0, 255);
  // change the analog out value:
  analogWrite(analogOutPin, outputValue);

  // print the results to the Serial Monitor:
  Serial.print("sensor = ");
  Serial.print(sensorValue);
  Serial.print("\t output = ");
  Serial.println(outputValue);

  // wait 2 milliseconds before the next loop for the analog-to-digital
  // converter to settle after the last reading:
  delay(2);
}
```

Figure 9. An Arduino code for an external LED and a pot to control the brightness of LED.

EXPERIMENT-5

ARDUINO-II

OBJECTIVES-

1. Design the Arduino Circuit to measure the temperature by using temperature sensor (LM35) such that for temperature $> 30^{\circ}\text{C}$ red led will glow and for temperature $< 30^{\circ}\text{C}$ yellow led will glow.
2. Design the Arduino Circuit display the 0-9 digits on seven segment.
3. Design the Arduino Circuit to measure the temperature by using temperature sensor (LM35) and display it on seven segment display (2 digits).

➤ **Objective-1.** Design the Arduino Circuit to measure the temperature by using temperature sensor (LM35) such that for temperature $> 30^{\circ}\text{C}$ red led will glow and for temperature $< 30^{\circ}\text{C}$ yellow led will glow.

- **Hardware required-** Arduino Board, USB cable, Breadboard, LM-35 IC, LED's, 220-ohm resistors.
- **Component description-** An LM35 is a 3-pin temperature sensor. It gives output in voltage according to the temperature, having an accuracy of 10mV per degree Celsius. It can measure temperatures in the range of -55°C to 150°C and it operates from 4V to 20V. Its output voltage is proportional to the temperature i.e., higher the temperature, higher is the output voltage. The output voltage of this sensor can be easily converted to digital form using an ADC so that a microcontroller can easily process it. Fig. 1 displays the pin diagram of LM-35 IC.
- **Connections-** For connections, refer Fig. 2 and the following steps-
 - i. Connect the Left pin of LM35 to the 5V of Arduino.
 - ii. Connect the center pin of LM35 to the A0 of Arduino.
 - iii. Connect the right pin of LM35 to the GND of Arduino.

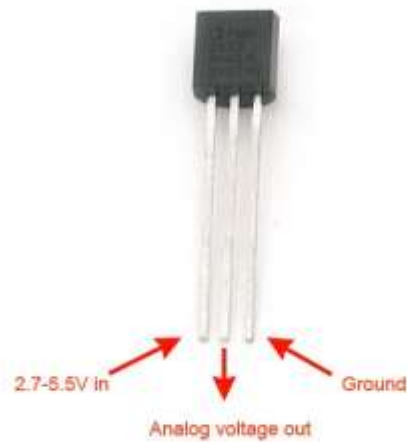


Fig. 1. Pin diagram of an LM-35 IC.

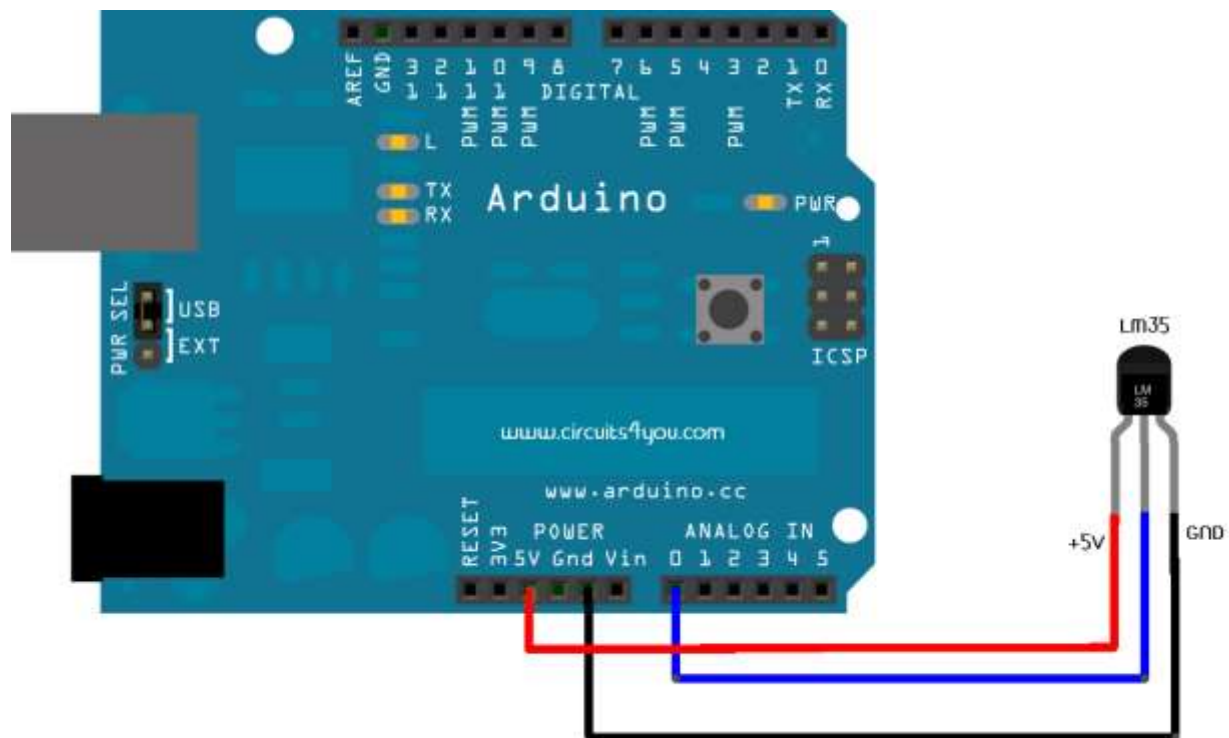


Fig. 2. Connections for the LM-35 temperature sensor with the Arduino.

- **Code-**

```
float temp;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  temp = analogRead(A0);
  temp = temp*0.48828125;
  Serial.print("TEMPERATURE: ");
  Serial.print(temp);
  Serial.print("*C");
  Serial.println();
  delay(1000);
}
```

➤ **Objective-2.** Design the Arduino Circuit display the 0-9 digits on seven segment.

- **Theory-** Seven segment displays are of two types: Common Cathode (CC) and Common Anode (CA). The Internal structure of both types is nearly the same. The difference is the polarity of the LEDs and common terminal. As their name suggests, the common cathode has all the cathodes of the LEDs in a 7-segment connected together and the common anode has all the anodes of the LEDs in a 7-segment connected together.

In the common cathode display, all the cathode connections of the LED segments are connected together to 'logic 0' / GND. The individual segments are then illuminated by applying a HIGH / 'logic 1' signal to the individual Anode terminals (a-g).

In the common anode display, all the anode connections of the LED segments are joined together to logic "1". The individual segments are illuminated by applying a ground, logic "0" or "LOW" signal to the Cathode of the particular segment (a-g)

- **How to know the type of the seven segment?**

You can know the type by testing it, when you connect one of the center pins from the seven segment to GND and connect one of the other pins to VCC and it turned on, that means the seven segment is common cathode, otherwise the seven segment is common anode.

- **How to know the pin?**

You can test them by connecting the center to (GND to cathode / VCC to anode) and any other pin to (VCC to cathode / GND to anode).

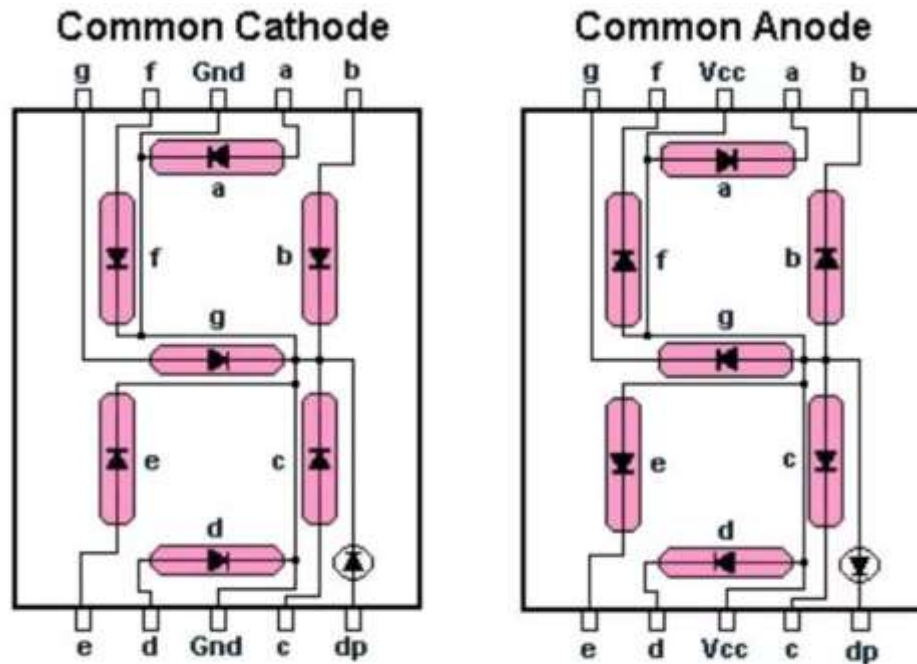


Fig. 3. Common cathode and common anode LED.

- Our task is to drive Seven Segment Display using Arduino, by giving decimal less than 10 through serial monitor of Arduino software.
- For that first we have to convert that number in 4-bit binary number.
- After converting it into 4-bit number, that number is used to write the equations of a, b, c, d, e, f and g (the pins of seven segment display) using K-Maps.
- Then feed this into Seven Segment Display through Arduino output pins, which glow each segment of the display according to the decimal number given as input in the serial monitor.

- **Hardware Required-**

- **Connection-**

- Pin g connected to 5
- Pin f connected to 4

- com connected to 12
- Pin a connected to 3
- Pin b connected to 2
- Pin e connected to 7
- Pin d connected to 8
- Pin c connected to 6

• **Code-**

```
#define A 3
#define B 2
#define C 6
#define D 8
#define E 7
#define F_SEG 4
#define G 5
// Pins driving common anodes
#define CA1 12
#define CA2 13
// Pins for A B C D E F G, in sequence
const int segs[7] = { A, B, C, D, E, F_SEG, G };
// Segments that make each number
const byte numbers[10] = { 0b1000000, 0b1111001, 0b0100100, 0b0110000, 0b0011001,
0b0010010,
0b0000010, 0b1111000, 0b0000000, 0b0010000 };
void setup() {
  pinMode(A, OUTPUT);
  pinMode(B, OUTPUT);
  pinMode(C, OUTPUT);
  pinMode(D, OUTPUT);
  pinMode(E, OUTPUT);
  pinMode(F_SEG, OUTPUT);
  pinMode(G, OUTPUT);
  pinMode(CA1, OUTPUT);
  pinMode(CA2, OUTPUT);
}
void loop() {
  for (int digit1=0; digit1 < 10; digit1++) {
    for (int digit2=0; digit2 < 10; digit2++) {
      unsigned long startTime = millis();
      for (unsigned long elapsed=0; elapsed < 600; elapsed = millis() - startTime) {
        lightDigit1(numbers[digit1]);
        delay(5);
        lightDigit2(numbers[digit2]);
        delay(5);
      }
    }
  }
}
```

```

}
}
}
}
void lightDigit1(byte number) {
digitalWrite(CA1, LOW);
digitalWrite(CA2, HIGH);
lightSegments(number);
}
void lightDigit2(byte number) {
digitalWrite(CA1, HIGH);
digitalWrite(CA2, LOW);
lightSegments(number);
}
void lightSegments(byte number) {
for (int i = 0; i < 7; i++) {
int bit = bitRead(number, i);
digitalWrite(segs[i], bit);
}
}
}

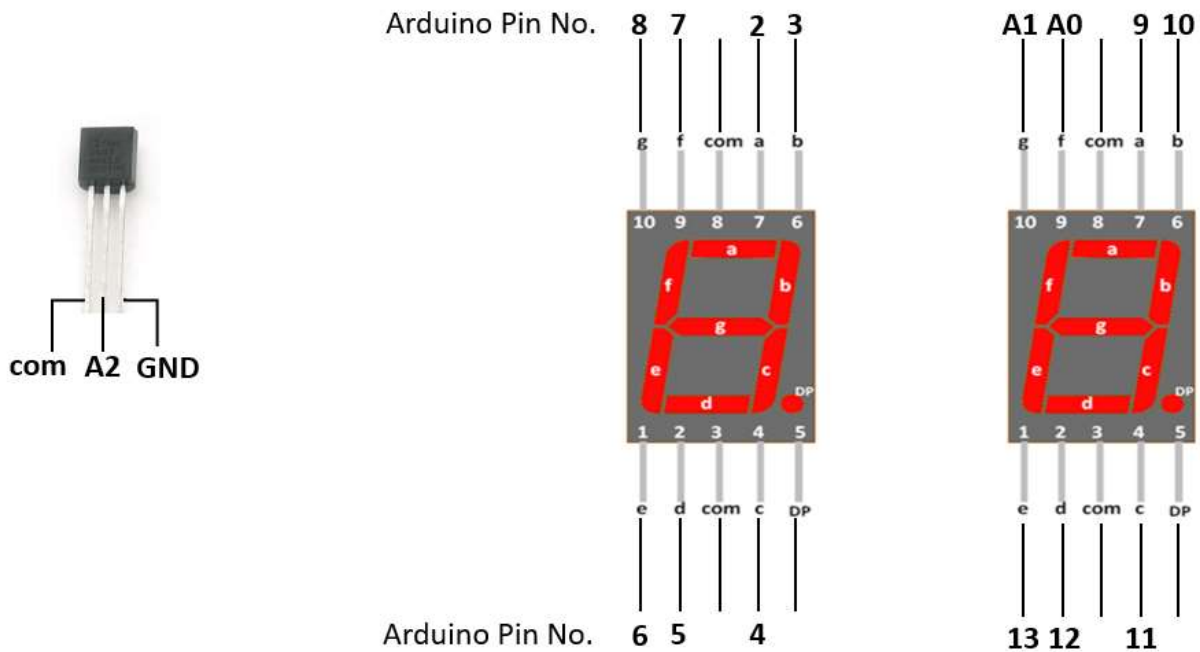
```

- **Objective-3.** Design the Arduino Circuit to measure the temperature by using a temperature sensor (LM35) and display it on seven segment display (2 digits).

HARDWARE REQUIRED- Arduino Board, USB cable, Breadboard, LM 35 IC, LEDs, 220 ohm resistors, two 7-segment displays.

COMPONENT DESCRIPTION –

Connections:



- All com pins (including that of the LM35) are shorted together and all DPs are left open.

- Connect both 7 segments with Arduino as shown in the above figure.
- All com pins (including the left pin of LM 35 as shown in the figure) are shorted together.
- Middle pin of LM 35 connected to A2 pin of Arduino.
- Right pin of LM 35 connected to GND pin of Arduino.
- Connect 500-ohm resistor, one terminal of the resistor to 5 V of Arduino pin and another terminal of the resistor connected to com pin of 7 segments.

Code:

```
int temp;
int tempPin = A2;
```

```

int x,y;
int bcd_array[10][7] = {
    { 0,0,0,0,0,0,1 }, // 0
    { 1,0,0,1,1,1,1 }, // 1
    { 0,0,1,0,0,1,0 }, // 2
    { 0,0,0,0,1,1,0 }, // 3
    { 1,0,0,1,1,0,0 }, // 4
    { 0,1,0,0,1,0,0 }, // 5
    { 0,1,0,0,0,0,0 }, // 6
    { 0,0,0,1,1,1,1 }, // 7
    { 0,0,0,0,0,0,0 }, // 8
    { 0,0,0,1,1,0,0 }}; // 9

```

```

void BCD0(int);
void BCD1(int);

```

```

void setup()
{
    pinMode(2, OUTPUT);
    pinMode(3, OUTPUT);
    pinMode(4, OUTPUT);
    pinMode(5, OUTPUT);
    pinMode(6, OUTPUT);
    pinMode(7, OUTPUT);
    pinMode(8, OUTPUT);
    pinMode(A2, INPUT);
    pinMode(9, OUTPUT);
    pinMode(10, OUTPUT);
    pinMode(11, OUTPUT);
    pinMode(12, OUTPUT);
    pinMode(13, OUTPUT);
    pinMode(14, OUTPUT);
    pinMode(15, OUTPUT);
}

```

```

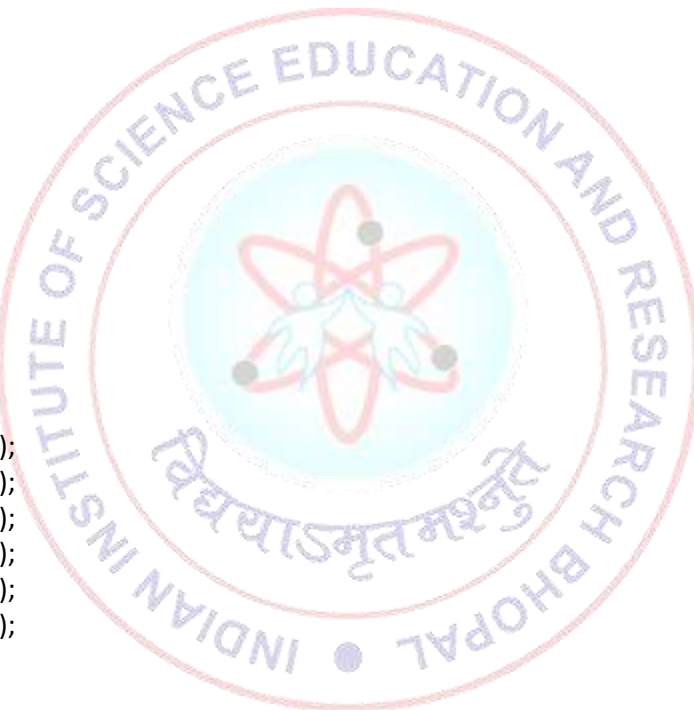
void loop()
{
    temp = analogRead(tempPin);
    temp = temp * 0.48828125;
    x = (temp/10);
    y = (temp -((temp / 10) * 10));
    delay(1000);
    BCD0(x);
    BCD1(y);
}

```

```

void BCD0(int number)
{

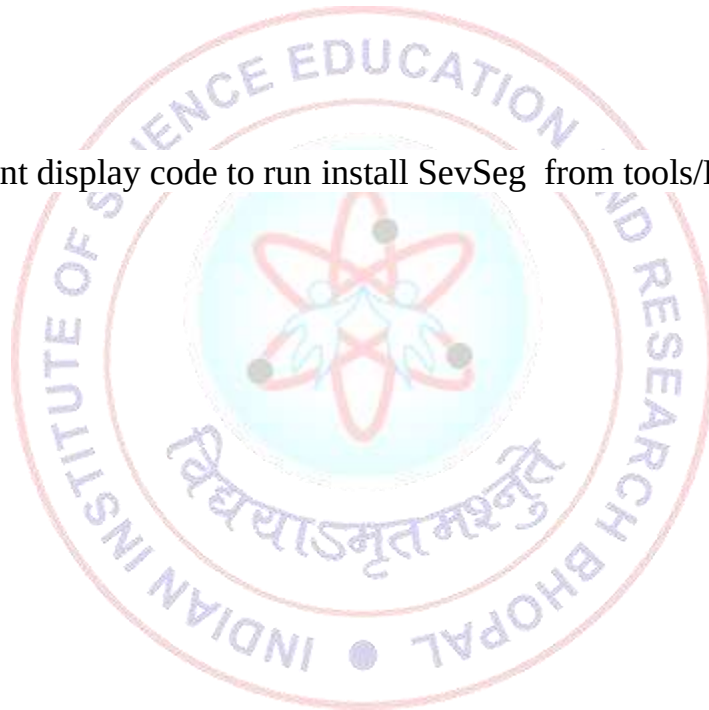
```



```
int pin= 2;  
for (int j=0; j < 7; j++) {  
    digitalWrite(pin, bcd_array[number][j]);  
    pin++;  
}  
}
```

```
void BCD1(int number)  
{  
    int pin= 9;  
    for (int j=0; j < 7; j++) {  
        digitalWrite(pin, bcd_array[number][j]);  
        pin++;  
    }  
}
```

Hint- for 7 segment display code to run install SevSeg from tools/Library Manager



EXPERIMENT No. 6

Objective

Design a circuit to decode BCD (binary coded decimal) to decimal number (0 to 9) using a seven-segment LED display.

Required components

- ✚ BCD to 7-segment LED IC (74LS47)
- ✚ 7-segment LED display
- ✚ Resistor $\times 7$ (220 Ω /470 Ω)
- ✚ Resistor $\times 4$ (1 k Ω)
- ✚ 5 V DC power supply
- ✚ Connecting wires
- ✚ Multimeter

Theory:

1. Seven Segment Display:

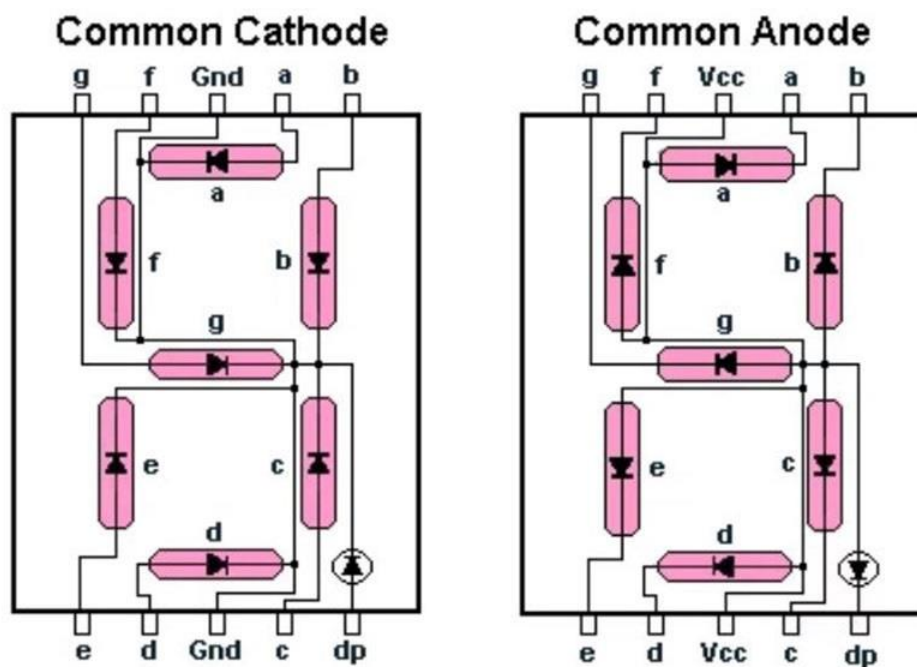


Fig. 3. Common cathode and common anode LED.

- Seven segment displays are of two types: Common Cathode (CC) and Common Anode (CA). The Internal structure of both types is nearly the same. The difference is the polarity of the LEDs and common terminal. As their name suggests, the common cathode has all the cathodes of the LEDs in a 7-segment connected together and the common anode has all the anodes of the LEDs in a 7-segment connected together.

In the common cathode display, all the cathode connections of the LED segments are connected together to 'logic 0' / GND. The individual segments are then illuminated by applying a HIGH / 'logic 1' signal to the individual Anode terminals (a-g).

In the **common anode** display, all the anode connections of the LED segments are joined together to **logic “1”**. The individual segments are illuminated by applying a ground, **logic “0” or “LOW” signal** to the Cathode of the particular segment (a-g)

- **How to know the type of the seven segment?**

You can know the type by testing it, when you connect one of the center pins from the seven segment to GND and connect one of the other pins to VCC and it turned on, that means the seven segment is common cathode, otherwise the seven segment is common anode.

- **How to know the pin?**

You can test them by connecting the center to (GND to cathode / VCC to anode) and any other pin to (VCC to cathode / GND to anode).

- Our task is to drive Seven Segment Display using Arduino, by giving decimal less than 10 through serial monitor of Arduino software.
- For that first we have to convert that number in 4-bit binary number.
- After converting it into 4-bit number, that number is used to write the equations of a, b, c, d, e, f and g (the pins of seven segment display) using K-Maps.
- Then feed this into Seven Segment Display through Arduino output pins, which glow each segment of the display according to the decimal number given as input in the serial monitor.

2. IC DM74LS47:

- The IC DM74LS47, takes 4 lines of BCD (Binary Coded Decimal) as input data (0000 to 1001 as 0 to 9), generates their complements internally, and decodes the data with seven outputs for a seven-segment display.
- The 7 outputs can be connected directly to the 7 inputs of the seven-segment display. Each output from the IC provides a signal Low to each input of the seven-segment display.
- The function of the decoder is as per the truth table given below.
- **Truth Table**

D	C	B	A	a	b	c	d	e	f	g	Display Number
0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	1	0	0	1	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0	2
0	0	1	1	0	0	0	0	1	1	0	3
0	1	0	0	1	0	0	1	1	0	0	4
0	1	0	1	0	1	0	0	1	0	0	5
0	1	1	0	1	1	0	0	0	0	0	6
0	1	1	1	0	0	0	1	1	1	1	7
1	0	0	0	0	0	0	0	0	0	0	8
1	0	0	1	0	0	0	1	1	0	0	9

Circuit Diagram

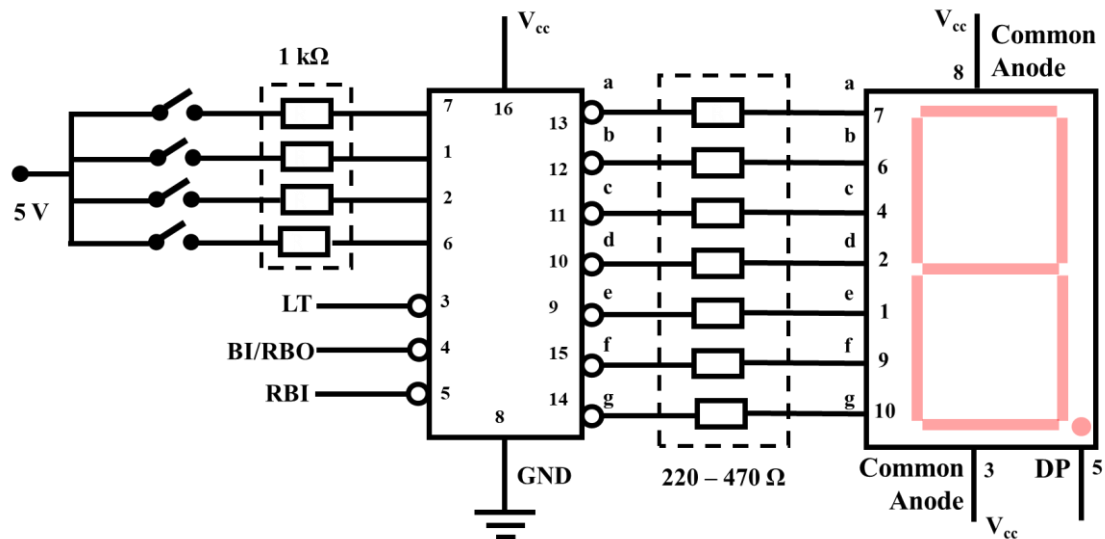


Fig. 1. Circuit diagram to decode binary to decimal using a seven-segment LED display

Procedure

- Set up the Circuit as shown in fig.1.
- Apply logic '0' level to LT and observe the seven segments of the LED. All the segments must be ON.
- Apply logic '0' level to BI/RBO and observe the seven segments of the LED. All the segments must be OFF.

- iv. Apply logic '1' to LT and RBI and observe the number displayed on the LED for all the inputs 0000 through 1111. This is the normal decoding mode.
- v. Apply logic '1' to LT and logic '0' to RBI, and observe the BI/RBO output and the number displayed on the LED for all the inputs 0000 through 1111. This is the normal decoding mode with zero blanking.

The functions of LT, RBI, RBO and BI are given below.

LT: This is called the LAMP TEST terminal and is used for segment testing. If it is connected to logic '0' level, all the segments of the display connected to the decoder will be ON. For normal decoding operation, this terminal is to be connected to logic '1' level.

RBI: For normal decoding operation, this is connected to logic '1' level. If it is connected to logic '0', the segment outputs will generate the data for normal 7-segment decoding, for all BCD inputs except Zero. Whenever the BCD inputs correspond to Zero, the 7-segment display switches off. This is used for zero blanking in multi-digit displays.

BI: If it is connected to logic '0' level, the display is switched-off irrespective of the BCD inputs. This is used for conserving the power in multiplexed displays.

RBO: This output is used for cascading purposes and is connected to the RBI terminal of the succeeding stage.

Expected Outcomes:

For any 4 bits input to the pins of IC DM74LS47 corresponding decimal number should be displayed on the seven-segment display.

Input Pins of IC DM74LS47:

Bits	Pin No.
A ₃	6
A ₂	2
A ₁	1
A ₀	7

Expected Output according to the inputs:

Inputs				Expected Output
A ₃ (6)	A ₂ (2)	A ₁ (1)	A ₀ (7)	Display
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9

Experiment 7: SR Latch

- **Objective 6.1:** Design SR flip-flop using NAND gates.
- **Objective 6.2:** Design SR flip-flop using NOR gates.
- **Objective 6.3:** Verify and test SR latch using SR latch IC.

Objective 6.1: Design SR flip-flop using NAND gates.

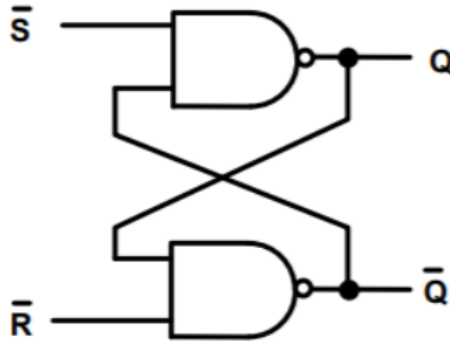
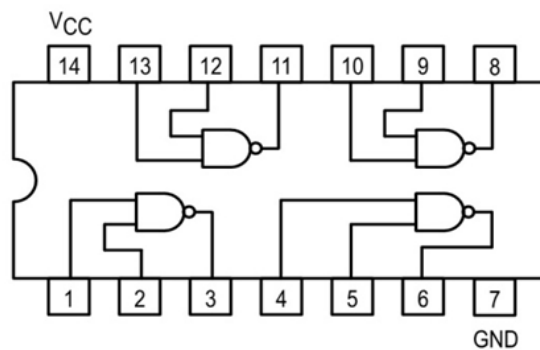


Figure 1: SR Latch with NAND gates

Equipment/Components Required:

S.No	Components	Specification values	Quantity
1	Breadboard		1
2	DC power supply	0-12 V	1
4	NAND gate IC	7400	1
7	LED		2
8	Connecting wires		As required

Pin Diagram:



Theory: This circuit requires prerequisite knowledge of the NAND gate.

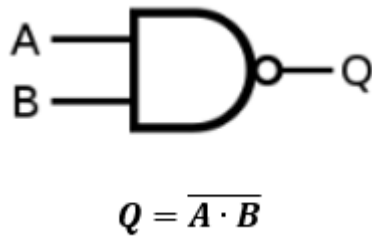


Figure 2: NAND gate

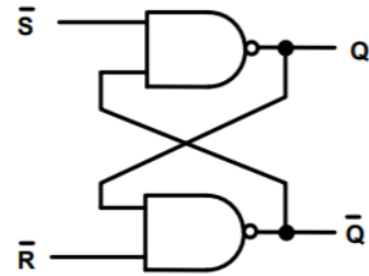


Figure 3: SR Latch with NAND gate

Truth Table:

Input		Output	
1	1		
0	1	1	0
1	0	0	1
0	0	x	x

Table 1: Truth table for SR latch with NAND gate

Procedure:

1. Connect the circuit as shown in the NAND gate circuit diagram.
2. Check the LEDs before connecting to the circuit.
(Connect anode to positive and cathode to negative terminals of the battery)
3. Don't increase the voltage by more than 5V.
4. For logic **1** as input, connect the respective terminal to +Vcc.
5. For logic **0** as input, connect the respective terminal to the Ground.

Result: Attach the images of the experimental circuit and label the LEDs.

Objective 6.2: Design SR flip-flop using NOR gates.

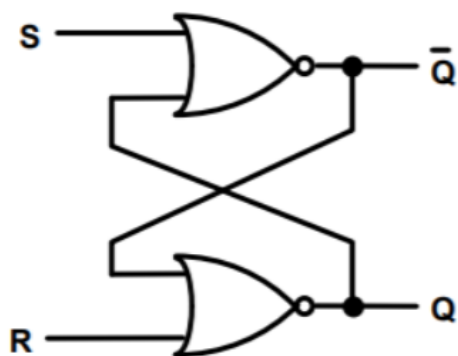
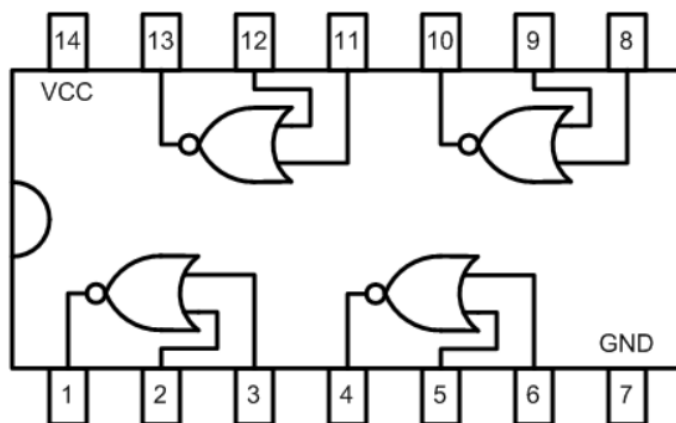


Figure 4: SR latch with NOR gate

Equipment/Components Required:

S.No	Components	Specification values	Quantity
1	Breadboard		1
2	DC power supply	0-12 V	1
4	NOR gate IC	7402	1
7	LED		2
8	Connecting wires		As required

Pin Diagram:



Theory: This circuit requires prerequisite knowledge of the NOR gate.

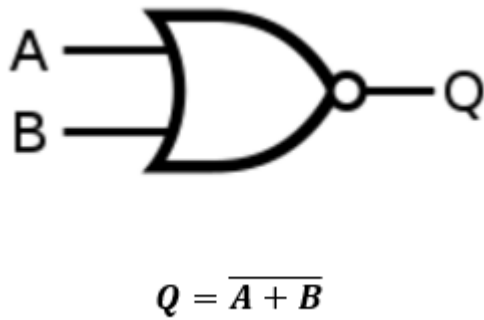


Figure 5: NOR gate

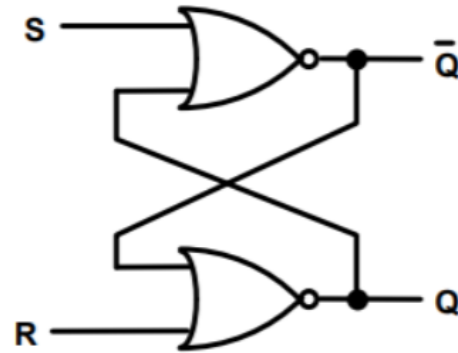


Figure 6: SR latch with NOR gate

Truth table:

Input		Output	
0	0		
0	1	0	1
1	0	1	0
1	1	x	x

Table 2: Truth table of SR latch with NOR gate

Procedure:

1. Connect the circuit as shown in the NOR gate circuit diagram.
2. Check the LEDs before connecting to the circuit.
(Connect anode to positive and cathode to negative terminals of the battery)
3. Don't increase the voltage by more than 5V.
4. For logic **1** as input, connect the respective terminal to +Vcc.
5. For logic **0** as input, connect the respective terminal to the Ground.

Result: Attach the images of the experimental circuit and label the LEDs.

Objective 6.3: Verify and test type1 and type2 SR latch using SR latch IC.

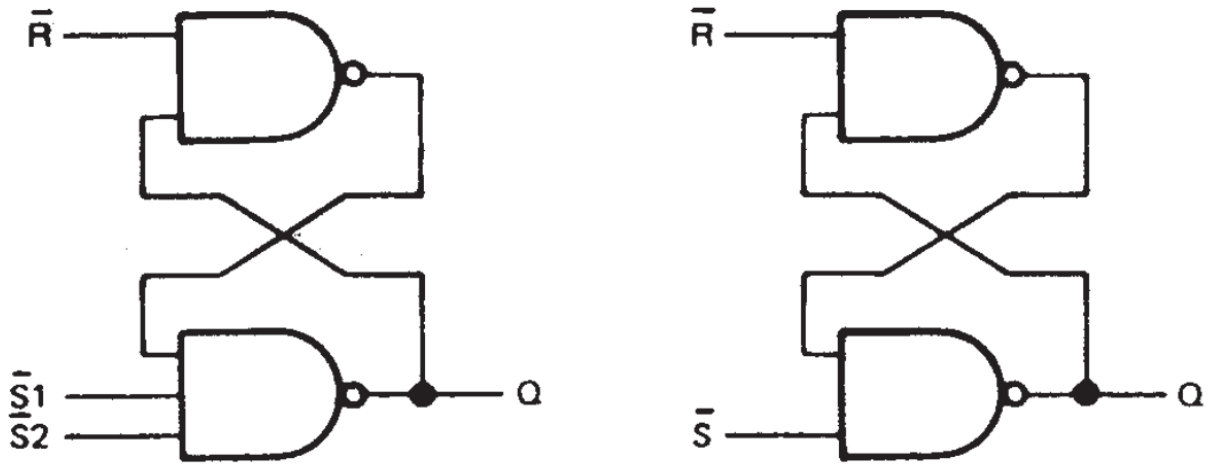
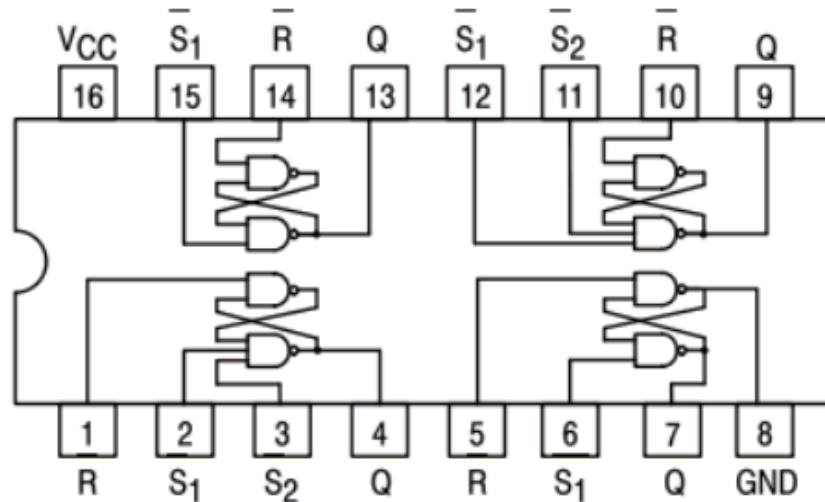


Figure 7: Type1 SR latch (left) and type2 SR latch (right)

Equipment/Components Required:

S.No	Components	Specification values	Quantity
1	Breadboard		1
2	DC power supply	0-12 V	1
4	SR latch IC	74LS279	1
7	LED		1
8	Connecting wires		As required

Pin Diagram:



Theory: This circuit requires prerequisite knowledge of the NAND gate. For the type1 and type2 SR latches the output is high as S1 and S2 are low, if all inputs are high then the output is indeterminate.

Truth table:

Input			Output
L	L	L	No Change(Q)
L	Don't care	H	H
Don't care	L	H	H
H	H	L	L
H	H	H	Pseudo stable

Table 3: Truth table for type1 SR latch

Input		Output
H	H	No Change(Q)
L	H	H
H	L	L
L	L	Pseudo stable

Table 4: Truth table for type2 SR latch

Procedure:

1. Connect the circuit as shown in the SR latch type1 and type2 circuit diagram.
2. Check the LEDs before connecting to the circuit.
(Connect anode to positive and cathode to negative terminals of the battery)
3. Don't increase the voltage by more than 5V.
4. For logic **1** as input, connect the respective terminal to +Vcc.
5. For logic **0** as input, connect the respective terminal to the Ground.

Result: Attach the images of the experimental circuit and label the LEDs.

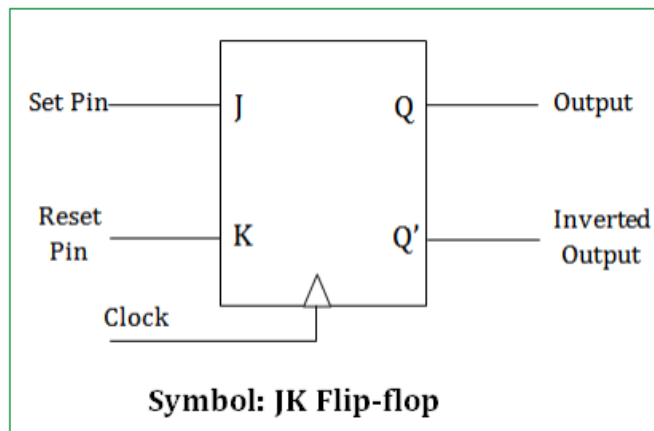
Experiment- 8

Theory:

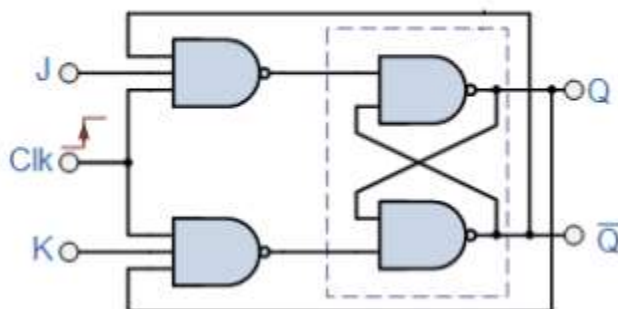
The JK flip-flop has two data inputs, J and K, and an EN/clock pulse input (CP). It has no undefined states, however. The fundamental difference of this device is the feedback paths to the AND gates of the input, i.e., Q is AND-ed with K and CP and Q' with J and CP. Whenever the clock signal is LOW, the input is never going to affect the output state. The clock has to be high for the inputs to get active. Thus, the JK flip-flop is a controlled Bi-stable latch where the clock signal is the control signal.

The JK flip-flop has the following characteristics:

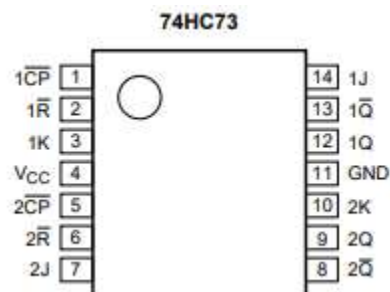
- If one input (J or K) is at logic 0, and the other is at logic 1, then the output is set or reset (by J and K, respectively), just like the RS flip-flop.
- If both inputs are 0, then it remains in the same state as it was before the clock pulse occurred. CP has no effect on the output.
- If both inputs are high, however, the flip-flop changes state whenever a clock pulse occurs; i.e., the clock pulse toggles the flip-flop again and again until the CP goes back to 0, as shown in the shaded rows of the characteristic table above. Since this condition is undesirable, it should be eliminated by an improvised.



Circuit Diagram:

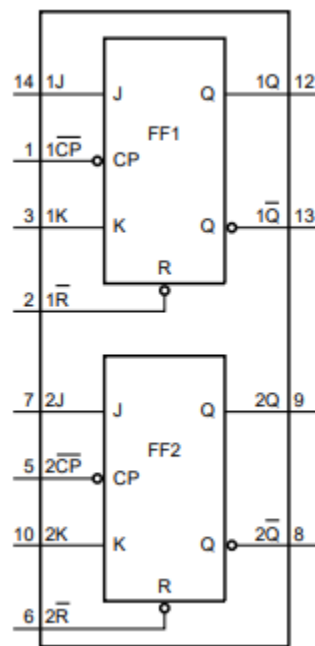
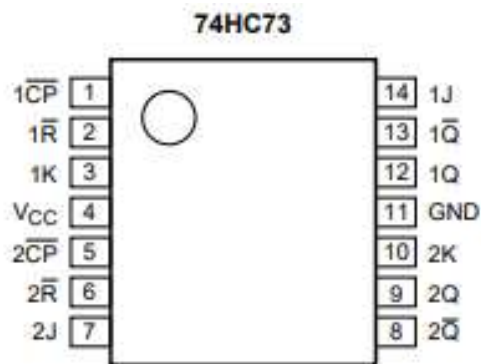


Pin Diagram and description:



Symbol	Pin	Description
1CP, 2CP	1, 5	clock input (HIGH-to-LOW edge-triggered); also referred to as nCP
1R, 2R	2, 6	asynchronous reset input (active LOW); also referred to as nR
1K, 2K	3, 10	synchronous K input; also referred to as nK
Vcc	4	positive supply voltage
GND	11	ground (0 V)
1Q, 2Q	12, 9	true output; also referred to as nQ
1Q, 2Q	13, 8	complement output; also referred to as nQ
1J, 2J	14, 7	synchronous J input; also referred to as nJ

Pin and functional Diagram:



Objectives: In this lab session we will learn about the working of JK flip flop and use it to create a frequency divider.

1. Familiarize yourself with the connection diagram, function table, and recommended operating conditions for the IC 74HC73N.
2. Verify the function table of the IC 74HC73N. Note down the outputs for various input settings.
3. Create frequency divider for $f/4$. That is, design a circuit such that, if the input clock frequency to the circuit is f , the circuit generates a square wave of frequency $f/4$. Record your output for input signals of different frequencies.

Procedure:

Objective 2.

1. Connect the circuit as shown in the circuit diagram.
2. Supply V_{cc} and ground from the power supply.
3. Provide a clock pulse using a square wave from the function generator.
4. Check the output from Q and $\bar{Q}$. Verify with the Function table .

Function table:

H = HIGH voltage level; h = HIGH voltage level one set-up time prior to the HIGH-to-LOW clock transition;

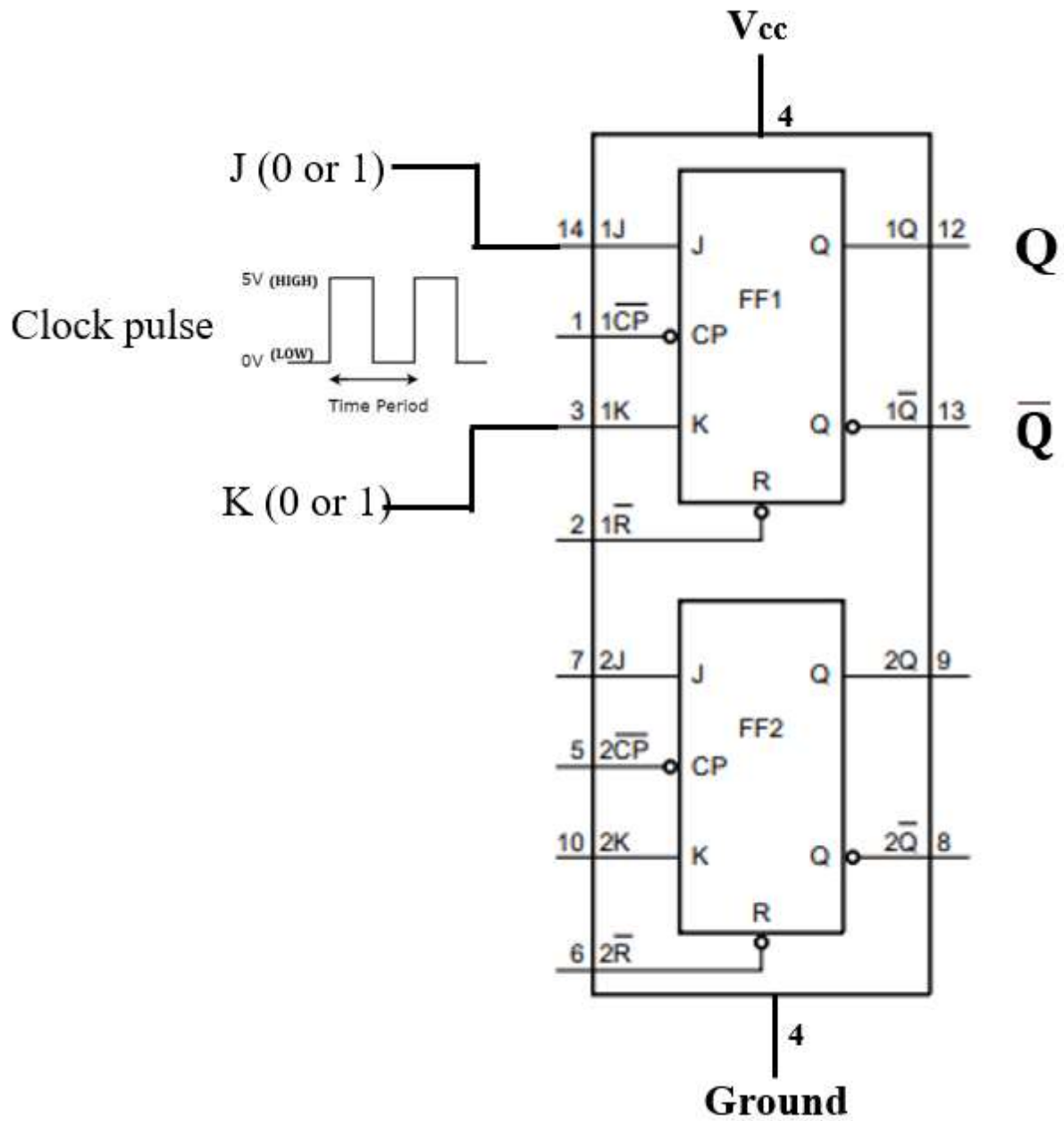
L = LOW voltage level; l = LOW voltage level one set-up time prior to the HIGH-to-LOW clock transition;

q = state of referenced output one set-up time prior to the HIGH-to-LOW clock transition;

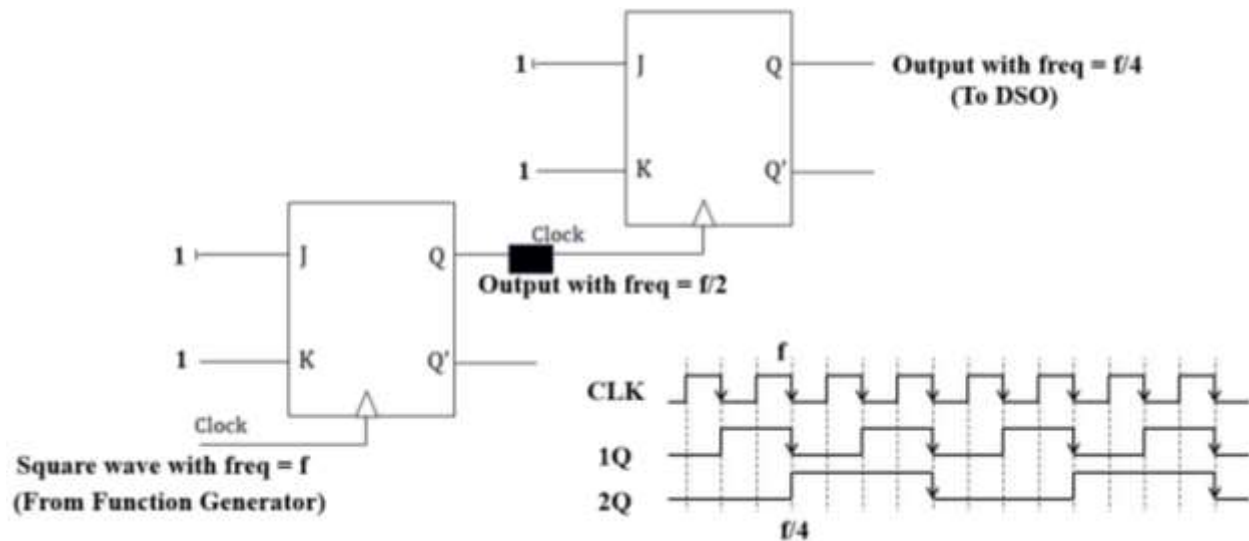
X = don't care; ↓ = HIGH-to-LOW clock transition.

Input				Output		Operating mode
nR	nCP	nJ	nK	nQ	n $\bar{Q}$	
L	X	X	X	L	H	asynchronous reset
H	↓	h	h	$\bar{q}$	q	toggle
H	↓	l	h	L	H	load 0 (reset)
H	↓	h	l	H	L	load 1 (set)
H	↓	l	l	q	$\bar{q}$	hold (no change)

Circuit Connection:

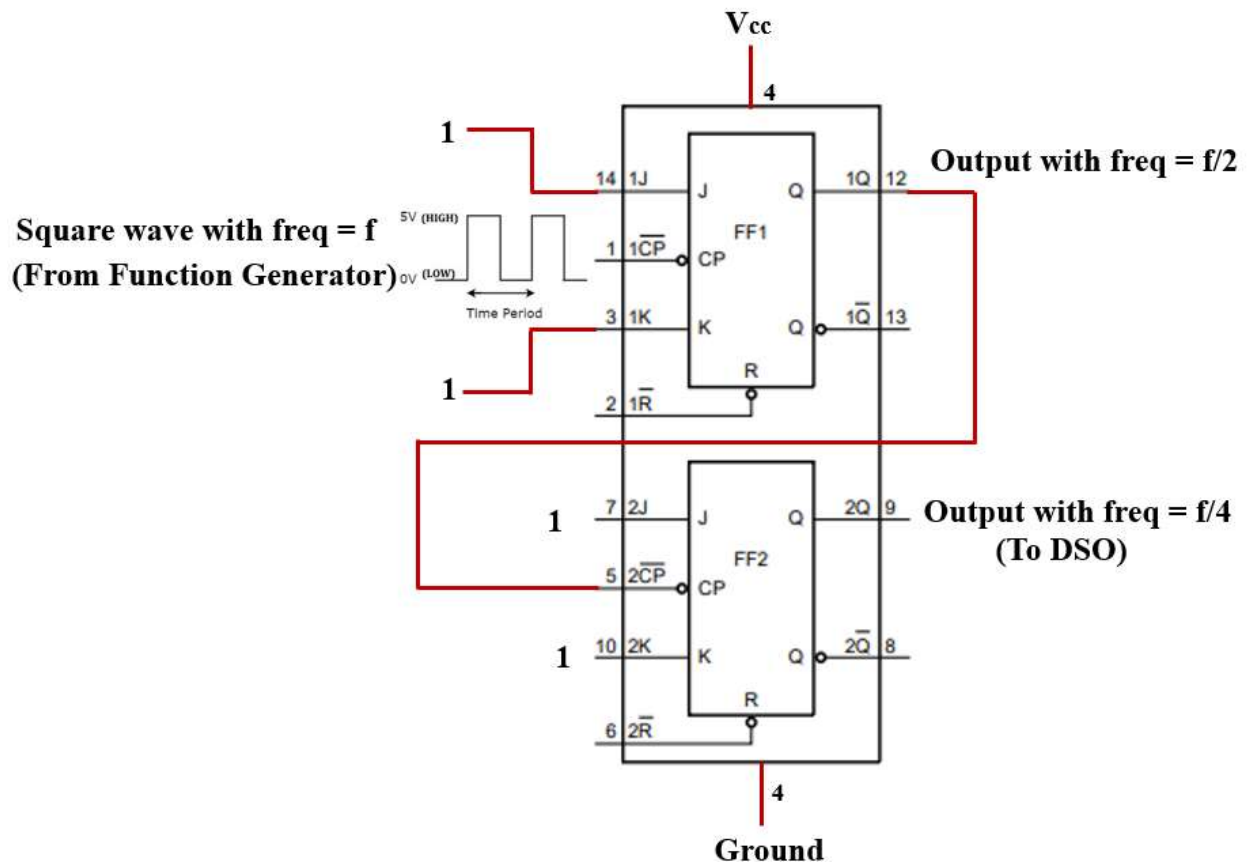


Objective 3. Frequency Divider



1. Connect the circuit as shown in the circuit diagram.
2. Supply V_{cc} and ground from the power supply.
3. J and K of both the flip flop should be high i.e. 1
4. Connect $1Q$ and $2\overline{CP}$.
5. Provide a clock pulse using a square wave from the function generator to $1\overline{CP}$ with frequency f .
6. Check the frequency of the output from 2Q pin by using DSO the frequency must be $f/4$.

Circuit Connection:



Experiment 9: Counters

Objective: Design counters using JK flip-flops.

Equipment/Components Required:

S.No	Components	Specification values	Quantity
1	Breadboard		1
2	DC power supply	0-12 V	1
3	JK flip-flop IC	74LS76	2
4	LED		2
5	Signal generator		1
6	Connecting wires		As required

Pin Diagram:

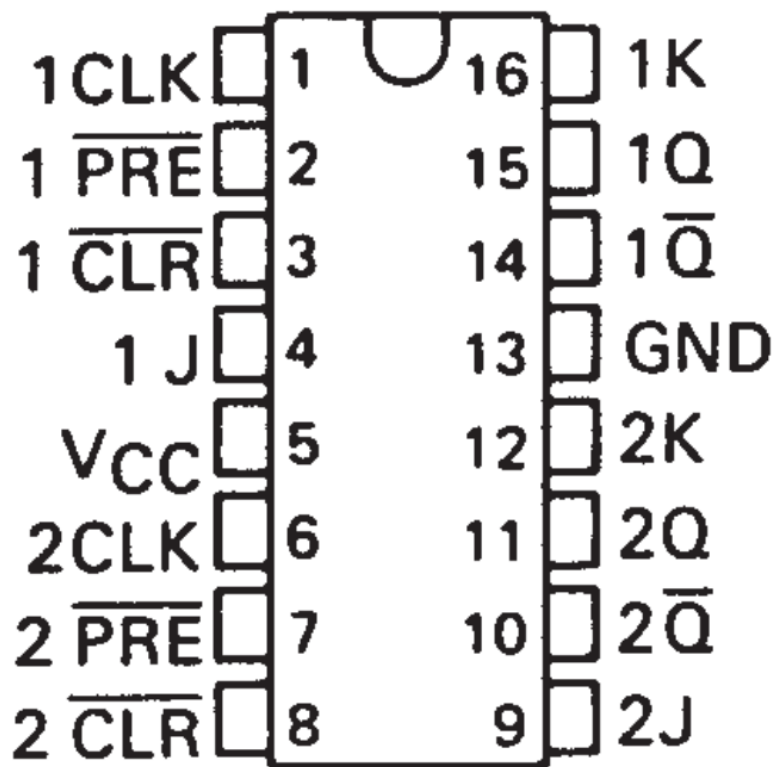


Figure 1: Pin diagram of 74LS76

Theory: This circuit requires prerequisite knowledge of the JK flip-flop.

Truth table				
Clock	J	K	Q	$\bar{Q}$
1	0	0	Q	$\bar{Q}$
1	0	1	0	1
1	1	0	1	0
1	1	1	Toggle state	

Characteristic table					
Clock	J	K	Q_n	Q_{n+1}	
1	0	0	0	0	
1	0	0	1	1	
1	0	1	0	0	
1	0	1	1	0	
1	1	0	0	1	
1	1	0	1	1	
1	1	1	0	1	
1	1	1	1	0	

Excitation table			
Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0
X=Don't care			

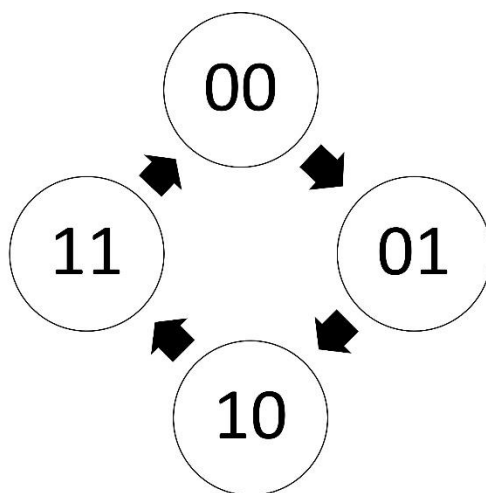


Figure 2: State diagram of 2-bit synchronous counter

Experimental circuit excitation table							
Present state		Next state		Output			
Q_1	Q_0	Q_1	Q_0	J_1	K_1	J_0	K_0
0	0	0	1	0	X	1	X
0	1	1	0	1	X	X	1
1	0	1	1	X	0	1	X
1	1	0	0	X	1	X	1

Karnaugh map:

The output after calculating K-map

$J_1 = Q_0$	$K_1 = Q_0$
$J_0 = 1$	$K_0 = 1$

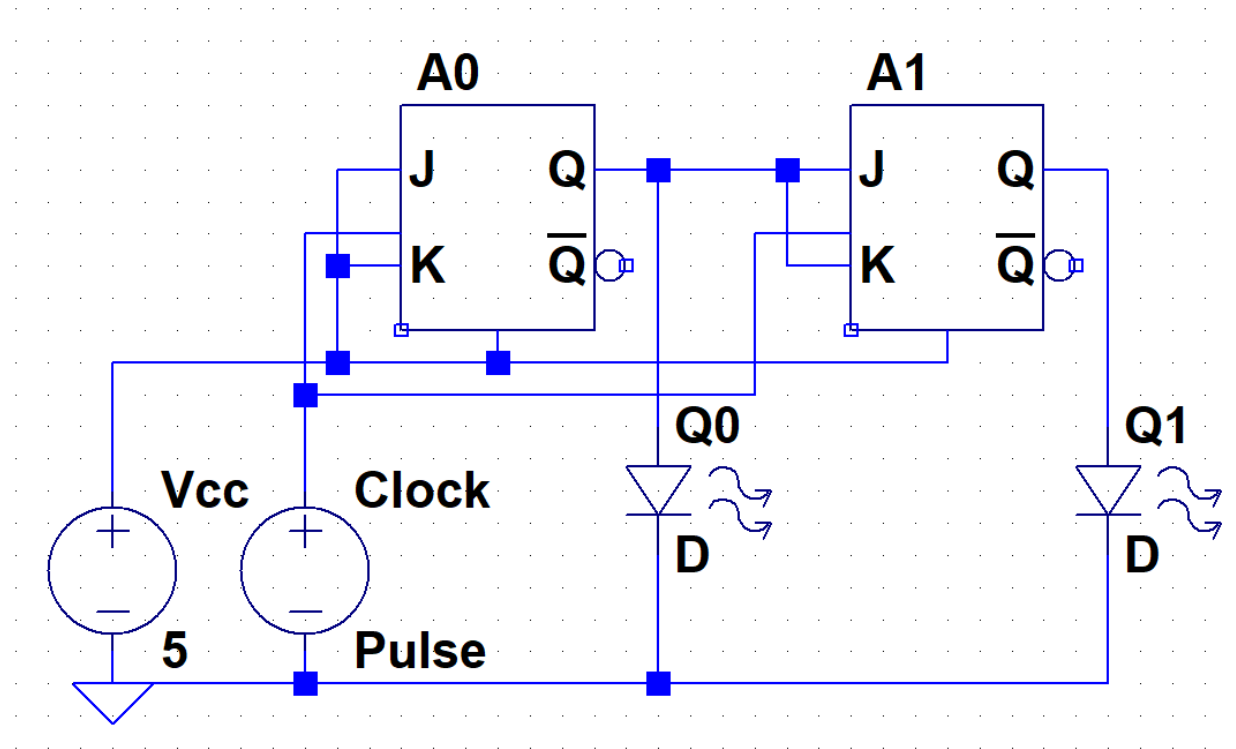
Experimental circuit diagram:

Figure 3: 2-bit synchronous counter using JK flip-flops

Procedure:

1. Connect the circuit as shown in the circuit diagram.
2. Check the LEDs before connecting to the circuit.
(Connect anode to positive and cathode to negative terminals of the battery)
3. Don't increase the voltage by more than 5V.
4. For logic **1** as input, connect the respective terminal to +Vcc.
5. For logic **0** as input, connect the respective terminal to the Ground.
6. Connect the PRE pins to logic 1.

Result: Attach the images of the experimental circuit and label the LED.